WNDRVR
SUPPORT NETWORK

VXWORKS 7 ITL_GENERIC BSP SUPPLEMENT GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 itl_generic BSP Supplement Guide, SR0650

12 August 2020

1. INTRODUCTION

1. Introducing the itl_generic BSP

The itl_generic BSP allows you to boot VxWorks on any Intel target board, and this document helps you to configure and use it correctly.

This document must be read in conjunction with the itl_generic BSP readme file that relates to your hardware board. For example, if you intend to boot VxWorks® on a generic Intel target board, you must refer to *itl_generic_readme.md*.

While the BSP readme guides you through the simplest target boot process, this document gives a more complete description of the BSP and its capabilities.

Before using a board with VxWorks, verify that the board runs in the factory configuration, and check all target board connections. Refer to the board vendor documentation for assistance.

Note: Helix Platform is tested on a subset of supported Intel boards. For more information, refer to the *Helix Platform Release Notes*.

Note: Wind River does not provide BIOS updates. Check with the board vendor.

2. Introducing the Boot Process

Booting an Intel target board into VxWorks requires a boot loader.

There are a variety of different boot mechanisms supported by this BSP. The preferred boot method will vary depending on the target board you have selected. Some targets provide a Compatibility Support Module (CSM) enabled BIOS that can load bootstrap code from storage devices such as a hard drive, an SD card, or a USB flash drive.

Other targets may provide a UEFI BIOS that does not have CSM support and so requires you to use an EFI loader (**BOOTX64.EFI**).

If you are more familiar with third-party loaders, such as GRUB or SYSLINUX, you can use those loaders with this BSP. Wind River does not support the loaders themselves, but does support the multiboot specification in VxWorks, making it possible for GRUB or SYSLINUX to load and run a VxWorks image on the target board.

Note: For information about booting VxWorks as a virtual machine rather than a native hardware target board, refer to Supported Virtual Machine Workflows on page 38.

3. Configure Development Environment

Initialise the workstation command shell for VxWorks board support package (BSP) configuration and project build.

If your workstation runs Windows, refer to Setting Up a Windows Development Environment on page 2 for instructions.

If your workstation runs Linux, refer to Setting Up a Linux Development Environment on page 2 for instructions.

Setting Up a Windows Development Environment on page 2 The Windows command-line interface must be configured correctly for use with VxWorks.

Setting Up a Linux Development Environment on page 2 The Linux command-line interface must be configured correctly for use with VxWorks.

3.1. Setting Up a Windows Development Environment

The Windows command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
C:\> cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
C:\WindRiver> wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright © 2009 Microsoft Corporation. All rights reserved.

C:\WindRiver>
```

3. Change into the Workbench workspace directory.

```
C:\WindRiver> cd workspace
C:\WindRiver\workspace>
```

Parent topic: Configure Development Environment on page 1

3.2. Setting Up a Linux Development Environment

The Linux command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

WNDRVR

Procedure

1. Change into the Wind River directory.

```
~/home/user$ cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
~/home/user/WindRiver$ ./wrenv.linux -p vxworks-7
~/home/user/WindRiver$
```

3. Create the workspace directory.

```
~/home/user/WindRiver$ mkdir workspace
~/home/user/WindRiver$
```

4. Change into the workspace directory.

```
~/home/user/WindRiver$ cd workspace
~/home/user/WindRiver/workspace$
```

Parent topic: Configure Development Environment on page 1

4. Configure Helix Development Environment

Initialise the workstation command shell for VxWorks board support package (BSP) configuration and project build for Helix.

If your workstation runs Windows, refer to Setting Up a Windows Development Environment for Helix on page 3 for instructions.

If your workstation runs Linux, refer to Setting Up a Linux Development Environment for Helix on page 4 for instructions.

Setting Up a Windows Development Environment for Helix on page 3
Setting Up a Linux Development Environment for Helix on page 4

4.1. Setting Up a Windows Development Environment for Helix

Before You Begin

Before you begin you must install the Helix Platform on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for Helix-VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
C:\> cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
C:\WindRiver> wrenv -p helix
Microsoft Windows [Version 6.1.7601]
Copyright © 2009 Microsoft Corporation. All rights reserved.

C:\WindRiver>
```

3. Change into the Workbench workspace directory.

```
C:\WindRiver> cd workspace
C:\WindRiver\workspace>
```

Parent topic: Configure Helix Development Environment on page 3

4.2. Setting Up a Linux Development Environment for Helix

Before You Begin

Before you begin you must install the Helix Platform on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for Helix-VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
~/home/user$ cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
~/home/user/WindRiver$ ./wrenv.linux -p helix
~/home/user/WindRiver$
```

3. Create the workspace directory.

```
~/home/user/WindRiver$ mkdir workspace
~/home/user/WindRiver$
```

4. Change into the workspace directory.

```
~/home/user/WindRiver$ cd workspace
~/home/user/WindRiver/workspace$
```

Parent topic: Configure Helix Development Environment on page 3

2. VXWORKS PROJECTS

1. Project Creation Workflow

A VxWorks Source Build (VSB) project must always be created before the VxWorks image project (VIP).

When creating a VSB using the itl_generic BSP, you must specify your project's intended micro architecture. If you omit this parameter, a 32-bit VSB will build for the **PENTIUM4** architecture and a 64-bit VSB builds for **CORE**. Wind River recommends you also specify the name of the Board Support Page (BSP) at VSB creation time, as this automatically configures the VSB with the necessary VxWorks layers.

From the command-line, you specify the BSP in the VSB creation command by adding -bsp itl_generic. Refer to Creating a VxWorks Source Build Specifying Only the Micro Architecture on page 8 for details on how to specify the CPU micro architecture at VSB build time.

If you do not specify the BSP name when creating the VSB, then you must then add the correct VxWorks layers manually to the project once it has been created. Refer to Creating a VxWorks Source Build Specifying Only the Micro Architecture on page 8 for details on how to determine which VxWorks layers must be added to the VSB.

Both the VSB and the VIP can be created using either the command-line interface, or Workbench. To learn more about VSB creation, refer to the VxWorks 7 Configuration and Build Guide guide.

The VIP is used to generate either a VxWorks boot application, or the VxWorks kernel image. For more details on the boot application, refer to the VxWorks Boot Loader User's Guide.

Note: The project creation workflow can also be achieved within the Workbench Eclipse IDE. For details, refer to *Workbench 4 Getting Started: Workbench Project Overview*

2. Processing Environments, Data Models and CPU Micro Architectures

You can choose between a variety of processing environments, data models and CPU micro architectures when building projects with the itl_generic board support package (BSP).

Processing Environments and Data Models

There are four variations supported by the itl_generic BSP:

- 32-bit Uniprocessor (UP)
- 32-bit Symmetric Multiprocessor (SMP)
- 64-bit UP
- 64-bit SMP

  Select your chosen configuration during creation of the VxWorks Source Build (VSB) project.

Table 1. Data Model VSB Options

Data Model Option	Parameter Specified During VSB Creation
32-bit Data Width	-ilp32 (default)
64-bit Data Width	-lp64
Uniprocessor	-up
Symmetric Multiprocessor	-smp (default)

Note: If no configuration is specified when creating the VSB, the project defaults to 32-bit data width, symmetric multiprocessor.

CPU Micro Architectures

When creating the VSB, the selection of the CPU micro architecture is done with the -cpu option.

The following CPU types are selected with -cpu:

- GOLDMONT
- AIRMONT
- ATOM
- BROADWELL
- CORE
- HASWELL
- IVYBRIDGE
- NEHALEM
- PENTIUM4
- SANDYBRIDGE
- SILVERMONT
- SKYLAKE
- WESTMERE

If no -cpu option is specified while creating 32-bit VSBs, the default **PENTIUM4** micro architecture is used. If the VSB is 64-bit, **CORE** is used instead.

Note: Neither the command line or Workbench facilities restrict any CPU and data model combinations. Invalid combinations, resulting in projects that do not build or function, are therefore possible. For example, **PENTIUM4 64-bit**.

3. Creating a Uniprocessor VxWorks Source Build

To use VxWorks in uniprocessor (UP) mode, a UP VxWorks source build (VSB) must first be created.

Before You Begin

Ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

About This Task

The recommended workflow when creating a UP VSB is to specify both the Board Support Package (BSP) and CPU microarchitecture at project creation time.

Procedure

1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -bsp itl_generic -cpu NEHALEM -up -lp64 vsb_NEH
ALEM_64_up -S
```

> Note: If the **NEHALEM** micro architecture is not correct for your board, change the value of the -cpu parameter. Refer to Processing Environments, Data Models and CPU Micro Architectures on page 5 for a list of alternative values. For example, -cpu SKYLAKE.

2. Build the VSB.

```
C:\WindRiver\workspace> cd vsb_NEHALEM_64_up
C:\WindRiver\workspace\vsb_NEHALEM_64_up> vxprj build
```

4. Creating a Symmetric Multiprocessor VxWorks Source Build

To use VxWorks in symmetric multiprocessor (SMP) mode, an SMP VxWorks source build (VSB) must first be created.

Before You Begin

Ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

About This Task

The recommended workflow when creating a symmetric multiprocessor (SMP) VxWorks Source Build (VSB) is to specify both the Board Support Package (BSP) and CPU microarchitecture at project creation time.

Procedure

1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -bsp itl_generic -cpu NEHALEM -smp -lp64 vsb_NE
HALEM_64_smp -S
```

> **Note:** If the **NEHALEM** micro architecture is not correct for your board, change the value of the -cpu parameter. Refer to Processing Environments, Data Models and CPU Micro Architectures on page 5 for a list of alternative values. For example, -cpu SKYLAKE.

2. Build the VSB.

```
C:\WindRiver\workspace> cd vsb_NEHALEM_64_smp
C:\WindRiver\workspace\vsb_NEHALEM_64_up> vxprj build
```

5. Creating a VxWorks Source Build Specifying Only the Micro Architecture

A VxWorks source build (VSB) can be created by specifying a particular Intel micro architecture.

Before You Begin

Before you begin, ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

About This Task

Creating a VxWorks source build (VSB) without explicitly specifying the board support package (BSP) requires you to identify and include the necessary layers into the project yourself.

Procedure

1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -cpu NEHALEM -smp -lp64 vsb_NEHALEM_64_smp -S
```

2. Add the VxWorks layers to the VSB to support the itl_generic BSP.

```
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add BOARDLIB
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add ITL_COMMON
```

3. Identify the additional VxWorks layers that must be added to the VSB to support the itl_generic BSP.

```
C:\WindRiver\workspace> cd vsb_NEHALEM_64_smp
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj bspInfo itl_generic
```

```
BSP=itl_generic_2_0_0_1 CPU=PENTIUM4 CORE AIRMONT ATOM BROADWELL GOLDMONST HASWELL IVYBRIDGE NE
HALEM SANDYBRIDGE SILVERMONT SKYLAKE WESTMERE ARCH=NEHALEM

...

LAYER_REQUIRES=(ACPICA)

SOFT_LAYER_REQUIRES=(USB_OHCI) && (USB_UHCI) && (USB_XHCI) && (USB_STORAGE) && (USB_KEYBOARD) &
& (EVDEV_DRV)

...
```

4. Add the specified VxWorks layers to the VSB.

```
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add ACPICA
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add USB_OHCI
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add USB_UHCI
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add USB_XHCI
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add USB_STORAGE
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add USB_KEYBOARD
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj vsb add EVDEV_DRV
```

5. Build the VSB.

```
C:\WindRiver\workspace\vsb_NEHALEM_64_smp> vxprj build
```

6. Boot Application Limitations

When building boot application projects, data model and compiler limitations exist.

1. Boot application projects, based upon a VxWorks Source Build (VSB) using the 32-bit data model, can only load 32-bit VxWorks Image Project (VIP) kernels onto the target board. Boot applications projects must be based upon a 64-bit VSB and can load both 32-bit and 64-bit VxWorks kernels onto the target board.
2. Only a boot application project built from a VSB using the itl_generic board support package (BSP) will be able to load VxWorks kernels also built from a VSB using the itl_generic BSP. The boot application cannot load VxWorks kernel files built from other VxWorks BSPs.
3. You compile the boot application project using the CLang compiler.

4 If you use **INCLUDE_MULTI_STAGE_WARM_REBOOT**, you must not initialize the boot device in the VxWorks boot application. If you intend to boot the target from a USB or SATA device, all USB or SATA related components must be removed from both the boot application project and the VxWorks VIP to be loaded by it. For example, if booting from a SATA device:

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip component remove PCI_DRV_STORAGE_
AHCI
```

5 If you use **INCLUDE_MULTI_STAGE_WARM_REBOOT** and **ACPI** together in either the boot application or the VxWorks VIP, make sure the target **BIOS** does not enable automatic handover to the operating system. **ACPI** touches all devices, whether or not drivers for the devices are loaded. When handoff to the operating system for **EHCI** is enabled, the **BIOS** loses control over **EHCI** the moment **ACPI** discovers the **EHCI** devices. VxWorks itself does a handoff from the **BIOS** to the operating system if USB device drivers are included, so the **BIOS** setting is not required. While most **BIOS** do not have this setting, if your **BIOS** does include it then change it as follows:

```
"Advanced"
    → "USB Configuration"
        → "BIOS EHCI Hand-Off"
            → Set to [DISABLED]
```

7. Creating a Boot Application Project

This board support package (BSP) supports the creation of a boot application project.

Before You Begin

These steps require the following:

- Your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.
- You have already created a uniprocessor VxWorks source build (VSB), **vsb_NEHALEM_64_up**.

Procedure

1. Create the boot application project.

```
C:\WindRiver\workspace> vxprj create itl_generic itl_generic_nehalem_bootapp -profile PROFILE_B
OOTAPP -vsb vsb_NEHALEM_64_up
C:\WindRiver\workspace> cd itl_generic_nehalem_bootapp
```

2. Select the multi-stage boot mode.

Option	Description
For multi-stage fast reboot mode	`C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip component add INCLUDE_FAST_REBOOT`
For multi-stage warm reboot mode	`C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip component add INCLUDE_MULTI_STAGE_WARM_REBOOT`

3. Set the boot application default boot line.

Option	Description
To boot from gei network device (Windows host)	`C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip parameter set DEFAULT_BOOT_LINE "\"gei(0,0)host:/path/name/vxWorks h=90.0.0.3 e=90.0.0.50 u=user"\"`
To boot from gei network device (Linux host)	`~/WindRiver/workspace/itl_generic_nehalem_bootapp# vxprj vip parameter set DEFAULT_BOOT_LINE '"gei(0,0)host:/path/name/vxWorks h=90.0.0.3 e=90.0.0.50 u=user"'`
To boot from the target file system (Windows host)	`C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip parameter set DEFAULT_BOOT_LINE "\"fs(0,0)host:/bd0/vxWorks h=90.0.0.3 e=90.0.0.50 u=user"\"`

Option	Description
To boot from the target file system (Linux host)	`~/WindRiver/workspace/itl_generic_nehalem_bootapp# vxprj vip parameter set DEFAULT_BOOT_LINE '"fs(0,0)host:/bd0/vxWorks h=90.0.0.3 e=90.0.0.50 u=user"'`

4. **Optional:** Add file system components.

Option	Description
To store boot parameters on a SATA or IDE disk	`C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip component add INCLUDE_SYS_WARM_AHCI`
To store boot parameters on a USB disk	`C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip component add INCLUDE_SYS_WARM_USB`

5. Build the boot application project.

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build set default_romCompress
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build vxWorks_romCompress
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build vxWorks_romCompress.bin
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build vxWorks_romCompress.pxe
```

6. Build the first stage loader.

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build vxStage1Boot.bin
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build vxStage1Boot.pxe
```

8. Deploying the Boot Application for the Multi-Stage Loader

Once built, the boot application must be deployed alongisde the multi-stage loader.

Before You Begin

These steps assume you have already installed the multi-stage loader to the boot device. Refer to Deploying the Boot Application for the Multi-Stage Loader on page 11 for more information.

Procedure

1. Copy the file **vxStage1Boot.bin** to your bootable media.
2. Rename **vxStage1Boot.bin** as **bootrom.sys**.
3. Copy the file **vxWorks_romCompress.bin** to your bootable media.
4. Rename **vxWorks_romCompress.bin** as **bootapp.sys**.

9. Deploying the Boot Application for PXE Boot

The boot application can optionally be deployed as part of a PXE boot environment.

Before You Begin

These steps assume you have already set up the PXE boot environment. Refer to Preboot Execution Environment (PXE) on page 26 for more information.

Procedure

1. Copy the file **vxStage1Boot.pxe** to your PXE server **TFTP** directory.
2. Rename **vxStage1Boot.pxe** as **bootrom.pxe**.
3. Copy the file **vxWorks_romCompress.pxe** to your PXE server **TFTP** directory.
4. Rename **vxWorks_romCompress.pxe** as **bootapp.sys**.

10. Deploying the Boot Application for the UEFI Boot Loader

The boot application can optionally be deployed as part of a UEFI boot environment.

Before You Begin

These steps assume you have already built the UEFI boot loader. Refer to Preboot Execution Environment (PXE) on page 26 for more information.

Procedure

1. Copy the UEFI loader file **BOOTX64.EFI** to your boot media at the path **/EFI/BOOT/BOOTX64.EFI**.
2. Copy the file **bootapp.sys** to your boot media at the path **/EFI/BOOT/BOOTAPP.SYS**.

11. VxWorks Image Project Notes

A number of important considerations must be taken into account when creating a VxWorks image project (VIP) .

1. All VIPs must be based on a pre-compiled VxWorks source build (VSB) project.
2. The multi-stage reboot component added to the VxWorks VIP must match the component added to the boot application project used to load VxWorks onto the target. The default selection is **INCLUDE_FAST_REBOOT**. You can change this to **INCLUDE_MULTI_STAGE_WARM_REBOOT**.
3. If you create a VIP using PROFILE_STANDALONE_DEVELOPMENT, then you can replace this profile with your preferred profile. For example, options are BSP_DEFAULT, PROFILE_DEVELOPMENT or no profile at all.
4. It is not essential that your VIP is created in your Workbench workspace directory. Creating the VSB in the workspace directory makes referencing this project in future VIPs easier.
5. You compile the VIP using the CLang compiler.

6. The **PROFILE_INTEL_GENERIC** profile can be used to create a VxWorks kernel image containing the most commonly used components and will boot successfully on a large percentage of Intel targets. This can be useful for initial board bring up and debugging. It is important to note the generic profile is not optimized for any particular target. An optimized image would only contain the components essential to the VIP for a specific target. Optimization is a subsequent step following a successful target boot of a generic VIP.
7. The **PROFILE_INTEL_GENERIC** profile configures the VIP with a target console on the video display, rather than the target serial port. You must select the correct video driver component based on the target BIOS setting. Refer to pcConsole Configuration on page 58 for details.
8. The simultaneous multi-threading (SMT) aware kernel scheduler is enabled by default in the VIP. You can disable this by removing the component **INCLUDE_SMP_SCHHED_SMT_POLICY** from the VIP. Refer to the VxWorks 7 Intel Architecture Guide for more details.

12. Creating a VxWorks Image Project

Creation of a VxWorks image project (VIP) involves important choices relating to target boot mode and target console device.

Before You Begin

These steps assume the following:

- Your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.
- You have already created a symmetric multiprocessor enabled VxWorks Source Build (VSB), **vsb_NEHALEM_64_smp**.

Procedure

1. Create the VxWorks image project (VIP):

```
C:\WindRiver\workspace> vxprj create itl_generic itl_generic_nehalem_kernel_smp -profile PROFIL
E_INTEL_GENERIC -vsb vsb_NEHALEM_64_smp
C:\WindRiver\workspace> cd itl_generic_nehalem_kernel_smp
```

2. Add the multi-stage boot mode component added by your boot application project:

Option	Description
For multi-stage fast reboot mode	`C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add INCLUDE_FAST_REBOOT`
For multi-stage warm reboot mode	`C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add INCLUDE_MULTI_STAGE_WARM_REBOOT`

3. **Optional:** If your kernel requires a kernel shell interface, add the following component bundle:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip bundle add BUNDLE_STANDALONE_S
HELL
```

4. Determine which video driver component must be used on this target. For details, refer to pcConsole Configuration on page 58.
5. If required, replace the M6845 video driver component with the EFI:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add DRV_CONSOLE_EFI
```

> **Note:** Alternatively, the VxWorks console can be directed to a target serial device instead of the video output. To achieve this, remove the component **INCLUDE_PC_CONSOLE**.

6. Build the VIP:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj build
```

7. Locate the VxWorks image file you have built at **C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp\default\vxWorks**.

13. Specifying the VxWorks and Boot Application Boot Device

The target boot device either loads the VxWorks kernel, or is the target network interface.

The boot device is specified as part of the default boot line. For example, the **GEI** network adapter is selected by the following example bootline. On Windows host:

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip parameter set DEFAULT_BOOT_LINE "\"g
ei(0,0)host:/path/name/vxWorks h=90.0.0.3 e=90.0.0.50 u=user"\"
```

On Linux host:

```
~/WindRiver/workspace/itl_generic_nehalem_bootapp$ vxprj vip parameter set DEFAULT_BOOT_LINE '"ge
i(0,0)host:/path/name/vxWorks h=90.0.0.3 e=90.0.0.50 u=user"'
```

A variety of boot devices are available to be used when configuring the VxWorks kernel VIP, or boot application project.

Table 1. Data Model VSB Options

Boot Device Name	Description
gei	Network adapter
fei	Network adapter
rtg	Network adapter
tei	Network adapter
fs	USB flash driver, SATA (AHCI) disk or the file system boot of a VxWorks image with a built-in symbol table

14. Helix Platform VxWorks Projects

When creating a VxWorks project for the Helix Platform, you must specify a micro architecture rather than the BSP name. Wind River recommends you use the **core** micro architecture, though other supported micro architectures can also be used.

Creating a VxWorks Source Build for the Helix Platform on page 15A VxWorks source build (VSB) for Helix specifies the core Intel micro architecture.
Creating a VxWorks Image Project for the Helix Platform on page 16 Creation of a VxWorks image project (VIP) involves important choices relating to target boot mode and target console device.

14.1. Creating a VxWorks Source Build for the Helix Platform

A VxWorks source build (VSB) for Helix specifies the core Intel micro architecture.

Before You Begin

Before you begin, ensure your VxWorks development environment is correctly configured. Refer to Configure Helix Development Environment on page 3.

About This Task

Creating a VxWorks source build (VSB) without explicitly specifying the board support package (BSP) requires you to identify and include the necessary layers into the project yourself.

Procedure

1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -cpu CORE -smp -lp64 vsb_CORE_64_smp -S
```

2. Add the VxWorks layers to the VSB to support the itl_generic BSP.

```
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add BOARDLIB
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add ITL_COMMON
```

3. Identify the additional VxWorks layers that must be added to the VSB to support the itl_generic BSP.

```
C:\WindRiver\workspace> cd vsb_CORE_64_smp
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj bspInfo itl_generic
```

```
BSP=itl_generic_2_0_0_1 CPU=PENTIUM4 CORE AIRMONT ATOM BROADWELL GOLDMONST HASWELL IVYBRIDGE NE
HALEM SANDYBRIDGE SILVERMONT SKYLAKE WESTMERE ARCH=NEHALEM

...

LAYER_REQUIRES=(ACPICA)

SOFT_LAYER_REQUIRES=(USB_OHCI) && (USB_UHCI) && (USB_XHCI) && (USB_STORAGE) && (USB_KEYBOARD) &
& (EVDEV_DRV)

...
```

4. Add the specified VxWorks layers to the VSB.

```
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add ACPICA
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add USB_OHCI
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add USB_UHCI
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add USB_XHCI
```

```
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add USB_STORAGE
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add USB_KEYBOARD
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj vsb add EVDEV_DRV
```

5. Build the VSB.

```
C:\WindRiver\workspace\vsb_CORE_64_smp> vxprj build
```

Parent topic: Helix Platform VxWorks Projects on page 14

14.2. Creating a VxWorks Image Project for the Helix Platform

Creation of a VxWorks image project (VIP) involves important choices relating to target boot mode and target console device.

Before You Begin

These steps assume the following:

- Your VxWorks development environment is correctly configured. Refer to Configure Helix Development Environment on page 3.
- You have already created a symmetric multiprocessor enabled VxWorks Source Build (VSB), **vsb_CORE_64_smp**.

Procedure

1. Create the VxWorks image project (VIP):

```
C:\WindRiver\workspace> vxprj create itl_generic itl_generic_core_kernel_smp -profile PROFILE_I
NTEL_GENERIC -vsb vsb_CORE_64_smp
C:\WindRiver\workspace> cd itl_generic_core_kernel_smp
```

2. Add the multi-stage boot mode component added by your boot application project:

Option	Description
For multi-stage fast reboot mode	`C:\WindRiver\workspace\itl_generic_core_kernel_smp> vxprj vip component add INCLUDE_FAST_REBOOT`
For multi-stage warm reboot mode	`C:\WindRiver\workspace\itl_generic_core_kernel_smp> vxprj vip component add INCLUDE_MULTI_STAGE_WARM_REBOOT`

3. **Optional:** If your kernel requires a kernel shell interface, add the following component bundle:

```
C:\WindRiver\workspace\itl_generic_core_kernel_smp> vxprj vip bundle add BUNDLE_STANDALONE_SHEL
L
```

4. Determine which video driver component must be used on this target. For details, refer to pcConsole Configuration on page 58.
5. If required, replace the M6845 video driver component with the EFI:

```
C:\WindRiver\workspace\itl_generic_core_kernel_smp> vxprj vip component add DRV_CONSOLE_EFI
```

Note: Alternatively, the VxWorks console can be directed to a target serial device instead of the video output. To achieve this, remove the component **INCLUDE_PC_CONSOLE**.

6. Build the VIP:

```
C:\WindRiver\workspace\itl_generic_core_kernel_smp> vxprj build
```

7. Locate the VxWorks image file you have built at **C:\WindRiver\workspace\itl_generic_core_kernel_smp\default\vxWorks**.

Parent topic: Helix Platform VxWorks Projects on page 14

3. TARGET BOOT WORKFLOWS

1. Supported Target Boot Workflows

Six distinct VxWorks boot workflows are available with the itl_generic BSP.

Table 1. Data Model VSB Options

Target Boot Workflow	Section
The target loads VxWorks from a multi-stage boot loader that runs from local disk.	Multi-Stage Boot Application on page 19
The target loads VxWorks from a multi-stage boot loader that loads to the target from the Preboot eXecution Environment (PXE).	Preboot Execution Environment (PXE) on page 26
The target loads VxWorks from a multi-stage boot loader that includes the BIOS boot loader.	BIOS Boot Loader on page 36
The target boots a multiboot loader (GRUB or SYSLINUX) and loads VxWorks.	GRUB Boot Loader on page 29
The target boots the UEFI loader, followed by the multi-stage boot loader and then VxWorks.	UEFI Boot Loader on page 31
The target boots the UEFI loader, followed by VxWorks.	UEFI Boot Loader on page 31

Boot Methods and the VxWorks Bootline on page 18 For VxWorks to boot correctly, you must set the VxWorks bootline when creating your VxWorks image project (VIP).

1.1. Boot Methods and the VxWorks Bootline

For VxWorks to boot correctly, you must set the VxWorks bootline when creating your VxWorks image project (VIP).

The target boot method is selected from the VxWorks bootline, configured by the component parameter **DEFAULT_BOOT_LINE**. For example on Windows host::

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip parameter set DEFAULT_BOOT_LINE "\"g
ei(0,0)host:/path/name/vxWorks h=90.0.0.3 e=90.0.0.50 u=user pw=user o= f="\"
```

On Linux host:

```
~/WindRiver/workspace/itl_generic_nehalem_bootapp$ vxprj vip parameter set DEFAULT_BOOT_LINE '"ge
i(0,0)host:/path/name/vxWorks h=90.0.0.3 e=90.0.0.50 u=user pw=user o= f="'
```

Assuming network boot, if no password is specified using the *pw* option, the target uses remote shell (RSH) protocol to retrieve the file. If a target password is specified, the target uses FTP. TFTP can be used instead of FTP if the flags are set to **0x80**.

If local filesystem boot is required, the target can boot from a block device such as a USB flash drive or SATA drive. If you want the network initialised as well, then set the other field to the ethernet device. For example on Windows host:

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip parameter set DEFAULT_BOOT_LINE "\"f
s(0,0)host:/bd0/vxWorks h=90.0.0.3 e=90.0.0.50 u=user pw=user o=gei f="\"
```

On Linux host:

```
~/WindRiver/workspace/itl_generic_nehalem_bootapp$ vxprj vip parameter set DEFAULT_BOOT_LINE '"fs(
0,0)host:/bd0/vxWorks h=90.0.0.3 e=90.0.0.50 u=user pw=user o=gei f="'
```

Parent topic: Supported Target Boot Workflows on page 18

2. Multi-Stage Boot Application

The multi-stage boot application is a Wind River technology, configured and used to load and run the VxWorks kernel.

Boot image size limitations require special considerations when booting VxWorks on Intel. The Intel architecture imposes a **576 KB** initial resident image file size. You must therefore use the multi-stage boot application to work around this size limitation because it boots the target in multiple stages.

The target initially executes the vxld loader program, located in the first sector of the active partition on the target boot device. The job of vxld architecture, boot image size limitations is to search the boot device for the file **bootrom.sys**. On locating this file, vxld loads and runs this executable.

The first-stage image (**vxStage1Boot.bin**) must be renamed **bootrom.sys** and copied into the first sector of the boot media. When this program runs it loads the larger second stage boot application image (**vxWorks_romCompress.bin**), renamed **bootapp.sys**.

The **bootrom.sys** first-stage image file is read into low memory at address **0x8000**, or *FIRST_TEXT_ADRS*, using the **int 0x13** BIOS interface. It then loads the larger, compressed second-stage image file **bootapp.sys** into target memory at the address pointed to by the BSP macro *ROM_TEXT_ADRS* and it begins executing code at the address of the symbol romInit( ) in 32-bit data mode.

The second-stage image file **bootapp.sys** then loads and decompresses the VxWorks image file to the address pointed to by the BSP macro *RAM_DST_ADRS* (or *RAM_LOW_ADRS* for boot application project builds) while in 32-bit data mode. Then it runs VxWorks.

The multi-stage boot application can operate in two different modes, and the mode is selected by including either the component **INCLUDE_FAST_REBOOT** or **INCLUDE_MULTI_STAGE_WARM_REBOOT**.

The component **INCLUDE_FAST_REBOOT** keeps a copy and a checksum of the boot application in target memory. It restores this copy when the VxWorks kernel reboots, and jumps to the address pointed to by the summation of the macros *FIRST_TEXT_ADRS* and *ROM_WARM_HIGH* (**0x8010**).

The component **INCLUDE_MULTI_STAGE_WARM_REBOOT** reloads the second-stage image file **bootapp.sys** from the target's boot device when the VxWorks kernel jumps to the address pointed to by the summation of the macros *FIRST_TEXT_ADRS* and *ROM_WARM_HIGH* (0x8010). No target RAM is used for the second-stage image file copy.

When the component **INCLUDE_MULTI_STAGE_WARM_REBOOT** is used, you must not use PXE boot to load the boot application onto the target. Also, you must not initialize the boot device in either the boot application of the VxWorks Image Project (VIP). The reboot entry point will jump back to 16-bit data mode, and load the second-stage loader image file again from the boot media. It expects the boot media to be in the same state as when the BIOS handoff occurred.

Note: You must always use the CLang compiler for a multi-stage boot because the first-stage source file **vxStage1Boot.s** is coded for use with the CLang compiler directives.

Note: When you build a multi-stage boot application project and a VxWorks kernel VIP, the same multi-stage boot component must be included in both projects.

Boot Device for the Multi-Stage Boot Application on page 20 Your boot device must be carefully prepared to use it with the multi-stage boot application.
Installing the Multi-Stage Boot Application from a Linux Workstation on page 20 A Linux workstation can be used to install the multi-stage boot application on the boot device.
Installing the Multi-Stage Boot Application from a Windows Host on page 24 A Windows workstation can be used to install the multi-stage boot application on the boot device.
Operation of the Multi-Stage Boot Application on page 26 The multi-stage boot application supports both warm and cold reboot types and a two stage boot process.

2.1. Boot Device for the Multi-Stage Boot Application

Your boot device must be carefully prepared to use it with the multi-stage boot application.

You can use a USB flash drive or a hard drive as the multi-stage boot application boot media. However, you must boot from a primary bootable partition on the primary media as seen by the PC BIOS.

Wind River recommends you use the PARTED utility to create the primary bootable partition on the boot media. The reason is that PARTED uses PC BIOS calls to create the drive master boot record (MBR).

The partition and file system on the disk can be configured as either **FAT12**, **FAT16** or the VxWorks proprietry **VXLONGNAMES** file system. The **vxld** loader does not support **FAT32**. If you require **FAT32**, you must create two partitions on your boot device. Make the primary partition to be **FAT16** and the secondary partition **FAT32**.

If you boot the target from a hard drive, you must create a primary partition that is no larger than 2 GB in size. You must format the partition in **FAT12** or **FAT16** and install the boot application to it.

Parent topic: Multi-Stage Boot Application on page 19

2.2. Installing the Multi-Stage Boot Application from a Linux Workstation

A Linux workstation can be used to install the multi-stage boot application on the boot device.

Before You Begin

These steps assume:

1. You have selected a USB flash drive as your target boot device, and you have inserted it into your Linux workstation.
2. You have installed the utilities parted and mformat on the Linux workstation. For example:

```
~/home/user$ sudo apt-get install parted
```

> Note: Wind River recommends using a USB flash drive from a major vendor with a capacity of **16 Gb**.

3. You have created and built a boot application VxWorks Image Project (VIP). For further details, refer to Creating a Boot Application Project on page 9.

Procedure

1. Identify the USB flash drive device and block size.
    1. Locate the USB flash drive device name.

```
~/home/user$ su root
# /sbin/parted -l
 Model: Kingston DataTraveler 2.0 (scsi)
  Disk /dev/sdb: 15.5GB
  Sector size (logical/physical): 512B/512B
  Partition Table: msdos

  Number   Start    End     Size   Type    File system  Flags
  1        16.4kB   16.8MB  16.8MB primary              boot, lba
```

Note: USB flash drives are usually prefixed **/dev/sdb** while partitions are named **/dev/sdb1**, **/dev/sdb2**, and so on. The prefixed used depends on the type of media. For example, an SD card device will be prefixed as **/dev/sd**.

2. Identify the USB flash drive sector size from the parted utility output and verify it is 512 bytes.

```
    Sector size (logical/physical): 512B/512B
```

Note: The sector size is 512 bytes. The loader vxld requires a sector size of **512** bytes. Different sector sizes may prevent the 16-bit vxld utility, located in the first sector of the active partition on the boot device, from running and loading the **bootrom.sys** file from the USB flash drive.

Note: If the output indicates the following error, then refer to Creating a New Disk Label from the Parted Utility on page 23 to resolve it.

```
Error: /dev/sdb: unrecognised disk label
```

2. Create a boot partition on the USB flash drive using the utility parted.
   1. Restart the parted utility.

```
# /sbin/parted /dev/sdb
```

Note: This example assumes a USB flash drive mounted as **/dev/sdb**.

   2. Dump out the USB flash drive partition structure.

```
(parted) unit B
(parted) print
Model: Kingston DataTraveller 2.0 (scsi)
Disk /dev/sdb: 15479597056B
Sector size (logical/physical): 512B/512B
Partition Table: msdos


Number Start    End        Size       Type    File System  Flags
1      16384B   16777727B  16761344B  primary              boot, lba
```

   3. Observe the start address in the parted output.

```
Model: Kingston DataTraveller 2.0 (scsi)
...
```

```
Number Start    End        Size       Type    File System  Flags
1      16384B   16777727B  16761344B  primary              boot, lba
```

4. Remove the partition table and create a new **msdos** partition table.

```
(parted) rm 1
(parted) mktable
New disk label type? msdos
(parted) mkpart
Partition type?  primary/extended?  primary
File system type?  [ext2]? fat16
Start?
End?
```

Note: Enter the start address. This must be less than (63*512 byte) due to MS-DOS mode limitations.

Note: Enter the end address. The partition size must be less than 2 GB due to the FAT16 limitations.

5. Run the **print** command again to dump out the USB flash drive partition drive structure.
6. Verify that the *Flags* setting is reported for partition 1 as *boot*. If the *boot* flag is absent, toggle the *boot* flag:

```
(parted) toggle 1 boot
```

7. Run the **print** command again to dump out the USB flash drive partition drive structure.

Verify that the MS-DOS partition table with a bootable FAT16 file system is defined. The lba flag may be missing and this is okay.

8. Exit **parted**.

```
(parted) quit
```

3. Add the **vxld** VxWorks loader to the USB flash drive.
   1. Locate the VxWorks loader program in the VxWorks installation directories and copy it to your boot application project directory.

```
# cd /home/user/WindRiver/vxworks-7/pkgs_v2/os/psl/intel/itl_x86_common/fw
# cp vxld.bin /home/user/WindRiver/workspace/itl_generic_nehalem_bootapp/default/.
```

   2. Edit the file **/etc/mtools.conf** and add a new entry for the USB flash drive.

```
drive u: file="/dev/sdb1"
```

   3. Change to the boot application directory.

```
# cd /home/user/WindRiver/workspace/itl_generic_nehalem_bootapp/default
```

   4. Rename the boot application project executable files.

```
# cp vxStage1Boot.bin bootrom.sys
# cp vxWorks_romCompress.bin bootapp.sys
```

   5. Copy the multi-stage boot application executables to the USB flash drive.

```
# mformat u: -B vxld.bin
```

Note: If mformat generates an error, then eject and remove and then replace the USB flash drive into the workstation. Re-run the mformat command as follows.

```
# mformat u: -B vxld.bin
mformat: Hidden (18) does not match sectors (32)

# mformat u: -H 18 -B vxld.bin
```

6. Complete the copy of the boot application project executable files to the USB flash drive.

```
# mcopy bootrom.sys u:
# mcopy bootapp.sys u:
# sync
```

4. Verify that the USB flash disk is configured correctly.
   1. Verify the master boot record on the first sector of the USB flash drive using the hexdump utility. This will be accessed by the BIOS at boot time.

   ```
   # hexdump /dev/sdb -C -n512
   ```

   2. Verify that **vxld.bin** is written to the first sector of partition 1 on the USB flash drive.

   ```
   # hexdump /dev/sdb1 -C -n512
   ```

5. Eject the USB flash drive from the workstation.

```
# eject /dev/sdb1
```

Creating a New Disk Label from the Parted Utility on page 23 You must create a new disk label from within the parted utility.

Parent topic: Multi-Stage Boot Application on page 19

Creating a New Disk Label from the Parted Utility

You must create a new disk label from within the parted utility.

Before You Begin

These steps assume you have inserted a USB flash drive into your workstation and it has mounted as **/dev/sdb**.

Note: If your drive name is not **/dev/sdb**, then substitute your drive name in the instructions below.

Procedure

1. As the root user, run the parted utility on your USB flash drive.

```
% su root
% /sbin/parted /dev/sdb
```

2. Select the option **mktable** and create a new disk label.
3. Select the option **msdos** as the new disk label type.

WNDRVR

4. Select the option **q** to exit parted.

Parent topic: Installing the Multi-Stage Boot Application from a Linux Workstation on page 20

2.3. Installing the Multi-Stage Boot Application from a Windows Host

A Windows workstation can be used to install the multi-stage boot application on the boot device.

Before You Begin

These instructions assume:

1. You must locate and install the third-party program mkbt.exe. This program is available publicly on the Internet. Install mkbt.exe to your boot application project directory.
2. You must find a file named **bootsect.bin** in your Wind River product installation. Navigate to *installDir* **/vxworks-7/pkgs_v2/os/board/intel/itl_generic**. Copy **bootsect.bin** to your boot application project directory.
3. You have selected a USB flash drive as your target boot device, and you have inserted it into your Windows workstation.

Procedure

1. Execute the diskpart utility from the administrator command terminal.

```
C:\> diskpart
```

2. List the disks attached to the workstation.

```
DISKPART> list disk


  Disk ###  Status         Size      Free     Dyn  Gpt
  --------  ------------   -------   -------  ---  ---
  Disk 0    Online          931 GB       0 B
  Disk 1    Online         1907 MB
```

3. Select the USB disk.

```
DISKPART> select disk 1

Disk 1 is now the selected disk.
```

4. List the disks again to very Disk 1 is selected.

```
DISKPART> list disk


  Disk ###  Status         Size      Free     Dyn  Gpt
  --------  ------------   -------   -------   ---  ---
  Disk 0    Online          931 GB      0 B
* Disk 1    Online         1907 MB
```

5. Clean **Disk 1**.

```
DISKPART> clean

Diskpart succeeded in cleaning the disk.
```

6. Create the primary partition.

```
DISKPART> create partition primary

Diskpart succeeded in creating the specified partition.
```

7. Select and activate the partition.

```
DISKPART> select partition 1

Partition 1 is now the selected partition.

DISKPART> active

Diskpart marked the current partition as active.
```

8. Format the disk as **fat16**.

```
DISKPART> format fs=fat

 100 percent completed.

Diskpart successfully formatted the volume.
```

9. Assign a drive letter and exit.

```
DISKPART> assign

Diskpart successfully assigned the drive letter or mount point.

DISKPART> exit
```

10. Identify the Windows assigned drive letter for the USB flash drive.
11. Navigate to the boot application project directory.

```
C:\ cd C:\WindRiver\workspace\itl_generic_nehalem_bootapp\default
```

12. Install **bootsect.bin** to the USB flash drive.

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp\default> mkbt -x bootsect.bin g:
```

> **Note:** This step assumes Windows has mounted the USB flash drive as drive letter g:.

13. Copy the remaining two multi-stage boot application executable files to the USB flash device.

```
C:\WindRiver\workspace\itl_generic_nehalem_bootapp\default> copy bootrom.sys g:
C:\WindRiver\workspace\itl_generic_nehalem_bootapp\default> copy bootapp.sys g:
```

> **Note:** The image file on the USB flash drive must be contiguous on the device or the first stage of the boot will fail. A contiguous image can be generated by saving all image files from the USB flash drive to a temporary location, deleting all images from the USB flash drive, and then copying each image to the USB flash drive.

Parent topic: Multi-Stage Boot Application on page 19

2.4. Operation of the Multi-Stage Boot Application

The multi-stage boot application supports both warm and cold reboot types and a two stage boot process.

The BIOS reads the boot device at the first sector of the active partition and locates the VxWorks loader (vxld) written there. The target executes vxld. This program loads and executes the first stage boot loader, **vxStage1Boot.bin** (renamed as **bootrom.sys**) which in turn loads the second stage boot application, **vxWorks_romCompress.bin** (renamed as **bootapp.sys**).

The second stage boot application has a command line interface, used to configure the target boot parameters and settings. It is capable of loading the VxWorks kernel VIP from either a boot device on the target, or from a remote network server.

When a warm target reboot occurs, the target returns to the second stage boot application. A cold reboot returns the target to the first stage.

```
(vxStage1Boot.bin) +-> (vxWorks_romCompress.bin) -+-> (vxWorks) kernel ->+
                   |   ^                           |                     |
                   |   |                           V                     V
                   |   + -----------(ctrl-x)----------+<-----(ctrl-x)--------+
                   |                                                         |
                   +<----------------------(cold)---------------------------+
```

Parent topic: Multi-Stage Boot Application on page 19

3. Preboot Execution Environment (PXE)

The Preboot Execution Environment (PXE) enables you to boot VxWorks onto the target over the network without requiring a target boot disk.

PXE is provided for use in three ways:

1. PXE is often supported by the target board BIOS. Usually, the BIOS supporting PXE is on a target board with integrated hardware ethernet ports.
2. PXE is often supported by standalone ethernet adapter cards. Many of the Intel Pro/100 cards have management features including PXE support. The VxWorks **fei** driver supports these cards.
3. If the PXE implementation in the BIOS or ethernet adapter is not sufficient to work with VxWorks, you can try the publicly available gPXE boot loader instead. The VxWorks multi-stage boot application has been tested with gPXE version 1.0.1+. For more details, refer to http://etherboot.org/wiki/).

PXE is a 16-bit, real mode environment. You use it to load the first stage of the VxWorks multi-stage boot application (**vxStage1Boot.bin**). Boot applications you load using PXE are limited because they must be able to start up in 16-bit, real mode and must also be less than 640 KB in memory size. While **vxStage1Boot.bin** meets this requirement, standalone VxWorks images may not.

PXE requires a DHCP and a TFTP server process to be running on the workstation it uses to boot the target board. When PXE starts up, it uses DHCP to discover both a target IP address and the path to the boot application executable file it must load into target board memory. PXE assumes the DHCP and TFTP server processes are running on the same workstation.

PXE does not permit the setting of the VxWorks boot line. You must therefore set the VxWorks boot line in the boot application project using the **DEFAULT_BOOT_LINE** kernel component.

The PXE environment provides a universal API for accessing the target ethernet device. VxWorks does not support this PXE ethernet API. This means that, while you can load PXE to load the multi-stage boot application, a native VxWorks ethernet driver supporting the target board's ethernet controller must already be configured into the VxWorks boot application in order for it to subsequently load and boot a VxWorks kernel image onto the target board.

Operation of the PXE on page 27 PXE can boot the VxWorks using the multi-stage boot application.
Using PXE to Load the Multi-Stage Boot Application on page 27 Your workstation must be configured correctly to boot VxWorks using PXE.

3.1. Operation of the PXE

PXE can boot the VxWorks using the multi-stage boot application.

To use the PXE with the VxWorks multi-stage boot application, the first stage of the boot application must be built to support PXE. The file name of this executable is **vxStage1Boot.pxe** (subsequently renamed to **bootrom.pxe**).

When the multi-stage boot application is loaded from a local disk, the vxld program in sector 0 of the disk is normally loaded by the target BIOS at address 0x7C00. The vxld program locates the first stage of the boot application, **vxStage1Boot.bin** (renamed **bootrom.sys**), and loads it at address 0x8000 in target memory.

However, when the first stage loader is loaded by PXE, **vxStage1Boot.pxe** (**bootrom.pxe**) has an extra block of padding prepended to it when it is first built. When PXE loads the file, it loads it to target memory at address 0x7C00 just like vxld does to a disk resident loader. The PXE file padding is implemented as a leading block of 1024 NOP instructions in **vxStage1Boot.pxe**. This padding forces the first stage loader entry point, called romInit( ), to fall at memory address 0x8000 just as if it had been loaded from a target disk.

The first stage of the boot application can use the PXE universal network device interface (UNDI) to load the second stage boot application executable. The system uses the PXE BIOS int 0x1A interface to determine whether a boot application executable has been loaded using PXE UNDI or not. If PXE UNDI has been used, then the **PXENV+** structure is retrieved. The offset from the beginning of **PXENV+** points to **!PXE** which is also retrieved. The system uses this information to make subsequent PXE PXENV_GET_CACHED_INFO API( ) calls which retrieve TFTP host and directory information. Finally, the loader calls PXENV_TFTP_READ_FILE( ) to load the second stage of the boot application, named **bootapp.sys**.

> Note: The PXE multi-stage boot application header memory map can only include the Usable memory type.

Parent topic: Preboot Execution Environment (PXE) on page 26

3.2. Using PXE to Load the Multi-Stage Boot Application

Your workstation must be configured correctly to boot VxWorks using PXE.

Before You Begin

These steps assume the following:

1. You have created a boot application project on your linux workstation called **itl_generic_nehalem_bootapp**.
2. You have both a DHCP and TFTP server installed on your linux workstation.

> Note: The following steps explain how to configure the open source ISC DHCP server process running on a FreeBSD system.

3. The target board is on the network **10.0.0.0/24**, the ethernet address is **00:00:E8:01:02:03** and the intended target IP address is **10.0.0.3**.
4. The workstation IP address is **10.0.0.1**.

5. Your VxWorks development environment is configured to use the command-line interface.

Procedure

1. Setup the **dhcpd.conf** file on the workstation.

```
# dhcpd.conf
#
# Sample PXE boot configuration file for ISC dhcpd
#

ddns-update-style none;

subnet 10.0.0.0 netmask 255.255.255.0 {
  range 10.0.0.3 10.0.0.3;
  deny unknown-clients;
  host vxWorks_pentium_target {
    hardware ethernet 00:00:e8:01:02:03;
  }

  option routers 10.0.0.1;
  option broadcast-address 10.0.0.255;
  filename "/bootrom.pxe";
  default-lease-time 600;
  max-lease-time 7200;
}
```

2. Start the DHCP server on the workstation.

```
~/home/user$ su
% dhcpd -cf /path/to/dhcpd.conf
```

Note: You must login as superuser to run this command.

3. Configure the TFTP service by adding the following line to the file **/etc/rc.conf**.

```
inetd_enable="YES"
```

4. Run the TFTP service.

```
% sh /etc/rc.d/inetd stop
% sh /etc/rc.d/inetd start
% exit
```

Note: In this step, you exit from your login as superuser.

5. Build the multi-stage boot application project to support PXE.

```
~/home/user$ cd ~/WindRiver/workspace/itl_generic_nehalem_bootapp
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp$ vxprj build set default_romCompres
s
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp$ vxprj build vxWorks_romCompress
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp$ vxprj build vxWorks_romCompress.bi
n
```

```
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp$ vxprj build vxWorks_romCompress.px
e
```

6. Populate the TFTP server directory **/tftpboot** with the multi-stage boot application executable files.

```
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp/default_romCompress$ cd default_rom
Compress
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp/default_romCompress$ cp vxStage1Boo
t.pxe bootrom.pxe
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp/default_romCompress$ cp vxWorks_rom
Compress.bin bootapp.sys
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp/default_romCompress$ cp bootrom.px
e /tftpboot/.
~/home/user/WindRiver/workspace/itl_generic_nehalem_bootapp/default_romCompress$ cp bootapp.sy
s /tftpboot/.
```

7. Power-up the target board and watch PXE load the multi-stage boot application..

```
CLIENT MAC ADDR: 00 00 E8 01 02 03 GUID: xxxxxxxx xxxx xxxx xxxx xxxxxxxxxxxx
CLIENT IP: 10.0.0.3 MASK: 255.255.255.0 DHCP IP: 10.0.0.1
GATEWAY IP: 10.0.0.1
TFTP...
```

8. Observe the VxWorks boot application startup and display the banner on the target console.
9. Use the VxWorks boot application command-line interface to set the VxWorks boot string appropriately.
10. Load the VxWorks kernel image from the VxWorks boot application.

Parent topic: Preboot Execution Environment (PXE) on page 26

4. GRUB Boot Loader

GRUB is a supported VxWorks boot loader.

VxWorks *GRUB* support adheres to the Multiboot Specification. The multiboot information is acquired from the BIOS and passed to the operating system.

When you use GRUB to boot VxWorks, the VxWorks image uses the default boot line specified in the GRUB file **grub.cfg** using the optional sysbootline parameter. Alternatively, you can set the boot line in the VxWorks VIP using the **DEFAULT_BOOT_LINE** component.

An example **grub.cfg** file, taken from *GRUB2* version 1.98, shows the use of the sysbootline parameter:

```
# The following lines configure both keyboard and serial port for input.
# Both VGA console and serial ports are used for output.
# The serial port is configured for 9600 baud.
serial --unit=0 --speed=9600 --word=8 --parity=no --stop=1
terminal_output serial console
terminal_input serial console

set default=0
set timeout=30
set root=(hd0,1)
```

```
menuentry "vxWorks 64-bit" {
 multiboot /images/vxWorks sysbootline:gei(0,0)host:vxWorks h=90.0.0.3 e=90.0.0.50 u=target
}
```

When a VxWorks image file is loaded using GRUB, you may need to perform additional memory configuration in order to use all the available target RAM. For more information, refer to Memory Auto-Size on page 89.

The VxWorks image booted using GRUB does not support warm boot. Any reboot, or selection of **CTRL-X** from the kernel shell, forces a target cold boot.

If you are using *EFI GRUB*, it must load the files **BOOTX64.EFI** or **BOOTIA32.EFI** from the boot device.

For more information on GRUB and the multiboot specification, refer to two sources:

1. http://www.gnu.org/software/grub/
2. http://www.gnu.org/software/grub/manual/multiboot/multiboot.html

Use GRUB With VxWorks on page 30GRUB must be deployed on the boot device to use it with VxWorks.
Operation of the GRUB Multiboot Loader on page 30 While it loads various kernel image types, GRUB does impose limitations on the VxWorks reboot logic.

4.1. Use GRUB With VxWorks

GRUB must be deployed on the boot device to use it with VxWorks.

For instructions on how to deploy and use the GRUB bootloader on an Intel target with VxWorks, refer to the VxWorks Boot Loader User's Guide.

Parent topic: GRUB Boot Loader on page 29

4.2. Operation of the GRUB Multiboot Loader

While it loads various kernel image types, GRUB does impose limitations on the VxWorks reboot logic.

The BIOS reads the boot device at sector zero and locates the multiboot loader. The target executes the loader, which loads and runs *GRUB*. The *SYSLINUX* loader also conforms to the multiboot specification.

GRUB then loads the *ELF* VxWorks image file (**vxWorks** or **vxWorks_romCompress**), or the VxWorks binary image (**vxWorks.bin** or **vxWorks_romCompress.bin**).

GRUB does not support warm reboot. All target reboots are treated as cold boot.

```
(SYSLINUX) --+-> (vxWorks_romCompress) boot application ->+-> (vxWorks) kernel ->+
           |                                             |                      |
           |                                             V                      V
           +<------------ (cold) -----------------------+------ (cold) --------+
```

```
(SYSLINUX) --+----> (vxWorks) kernel ---->+
             |                            |
             |                            V
             +<-------- (cold) -----------+
```

Parent topic: GRUB Boot Loader on page 29

5. UEFI Boot Loader

The Wind River UEFI boot loader can boot VxWorks.

The UEFI boot loader file must be located on the boot device, and the file location is dictated by the *UEFI Specification*. For example, locate the loader at **/EFI/BOOT/BOOTX64.EFI**.

The UEFI boot loader can be used to load either the boot application, **vxWorks_romCompress**, or the VxWorks kernel image, **vxWorks**. The file can be either in binary or ELF format. The boot application, or VxWorks image file, must be located in the same directory as **BOOTX64.EFI** and the file must be renamed **bootapp.sys**. For example, locate the VxWorks image file as **/EFI/BOOT/bootapp.sys**.

The VxWorks bootline can be defined in the boot application or the VxWorks VIP by defining the **DEFAULT_BOOT_LINE** component.

Alternatively, you may set the boot line in the UEFI boot loader. This requires the macro **EXTRA_DEFINE** to be set to the value **-DBOOT_CONFIG** in the UEFI boot loader configuration file **defs.boot**. The VxWorks bootline is then specified on the target in the file **/EFI/BOOT/bootapp.cfg**. This file must only contain a single line specifying the bootline. The format of the line is as follows:

```
bootline:<VxWorks bootline>
```

For example, the file **/EFI/BOOT/bootapp.cfg** could contain the following:

```
bootline:gei(0,0)host:/path/name/vxWorks h=90.0.0.3 e=90.0.0.50 u=user
```

For more information about using the **UEFI Boot Loader** to boot the VxWorks kernel, change to the directory **C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi** and refer to the UEFI *README.txt* file.

Building and Deploying the UEFI Boot Loader on page 31 The UEFI boot loader must first be built before it is deployed on the boot device.
Installing the UEFI Boot Loader as the Default UEFI Boot Option on page 32 You must use the target BIOS to set the *UEFI* boot loader executable file, **BOOTX64.EFI**, as the default boot option.
Operation of the UEFI Boot Loader on page 35 The UEFI boot loader supports multiple VxWorks kernel types, along with warm and cold reboot.

5.1. Building and Deploying the UEFI Boot Loader

The UEFI boot loader must first be built before it is deployed on the boot device.

Before You Begin

Before you begin, ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

These steps also assume you deploy the *UEFI boot loader* on a USB flash drive, mounted by the workstation as **E:**.

Procedure

1. Change to the UEFI directory.

```
C:\WindRiver\workspace> cd ..\vxworks-7\pkgs_v2\boot\uefi
C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi>
```

2. Build the UEFI boot loader.

```
C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi> make
```

3. Locate the UEFI boot loader executable file that you have built.

```
C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi> cd C:\WindRiver\vxworks-7\workspace
C:\WindRiver\vxworks-7\workspace> cd uefi_x86_64
C:\WindRiver\vxworks-7\workspace\uefi_x86_64> dir BOOTX64.EFI

 Directory of C:\WindRiver\vxworks-7\workspace\uefi_x86_64

02/09/2018  15:09           118,272 BOOTX64.EFI

C:\WindRiver\vxworks-7\workspace\uefi_x86_64>
```

Note: The file **C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi\defs.boot** is used to set build time options for the UEFI boot loader. Instructions of build time option use are available within the file.

Note: Detailed documentation and examples of UEFI boot loader use are available in the **C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi\README.txt** and **C:\WindRiver\vxworks-7\pkgs_v2\boot\examples**.

4. Create the UEFI directory on the USB flash drive.

```
C:\WindRiver\vxworks-7\workspace\uefi_x86_64> mkdir E:\EFI E:\EFI\BOOT
```

5. Deploy the UEFI boot loader executable to the USB flash drive.

```
C:\WindRiver\vxworks-7\workspace\uefi_x86_64> copy BOOTX64.EFI E:\EFI\BOOT
```

Parent topic: UEFI Boot Loader on page 31

5.2. Installing the UEFI Boot Loader as the Default UEFI Boot Option

You must use the target BIOS to set the *UEFI* boot loader executable file, **BOOTX64.EFI**, as the default boot option.

Before You Begin

These steps require that you have built the UEFI boot loader, deployed it on a USB flash drive and inserted the USB flash drive into the target.

Procedure

1. Power up the target and enter the EFI shell from the **UEFI BIOS**.
2. Run the UEFI boot loader from the EFI shell:

```
Shell> fs0:

fs0:\> EFI\BOOT\BOOTX64.EFI
```

3. At the UEFI boot loader count down, press a key to intercept the boot loader sequence:

```
    Copyright 2012-2017 Wind River Systems



VxWorks UEFI Stage 1 Bootstrap (64-bit version)
Built on: Oct 20 2014/08:38:07


ACPI 2.0 table at: 00000000CAEABF98


Loading from disk...
Opening [\EFI\BOOT\bootapp.cfg]...
Opening file [\EFI\BOOT\bootapp.cfg] failed: Not Found
Opening [\EFI\BOOT\bootapp.sys]... Read 5432596 bytes.


Loading 64-bit ELF image.


Total memory: 8379342848 bytes


Booting VxWorks.
Press a key to abort: 3
Paused, press a key to continue... (press 'M' to enter the boot manager)...
```

4. Enter the UEFI utility boot manager by pressing **M**, and select option **3)** to create a boot path:

```
Press 'M' to enter the UEFI utility boot manager, and press 3) to create a boot path:


  VxWorks UEFI boot manager options:


  1) Return to loader
  2) Reset system
  3) Create boot path
  4) Delete boot path


  Select option: 3
```

5. Choose the boot option for your target. Usually, this is **entry [0]**.

```
  The following block devices are available as file system boot paths:


  [0]:[Acpi(PNP0A03,0)/Pci(1D|0)/Usb(5)/HD(Part1,Sig00000000)]


  Select a path as the default to boot VxWorks [0-0]:0
```

```
Boot variable Boot0000 is available.

Boot path added successfully.


Press a key to continue...
```

6. Press any key and select the option **2) Reset system**:

```
VxWorks UEFI boot manager options:


1) Return to loader

2) Reset system

3) Create boot path

4) Delete boot path


Select option: 2

Resetting...
```

7. Watch the target reboot and run the UEFI boot loader

```
Copyright 2012-2017 Wind River Systems

VxWorks UEFI Stage 1 Bootstrap (64-bit version)
Built on: Oct 20 2014/08:38:07


ACPI 2.0 table at: 00000000CAEABF98


Loading from disk...

Opening [\EFI\BOOT\bootapp.cfg]...
Opening file [\EFI\BOOT\bootapp.cfg] failed: Not Found
Opening [\EFI\BOOT\bootapp.sys]... Read 5432596 bytes.


Loading 64-bit ELF image.


Total memory: 8379342848 bytes


VGA text mode support available.


Booting VxWorks.

Press a key to abort: 1
```

```
Jumping to VxWorks image at 0x408000...
```

Parent topic: UEFI Boot Loader on page 31

5.3. Operation of the UEFI Boot Loader

The UEFI boot loader supports multiple VxWorks kernel types, along with warm and cold reboot.

Using the UEFI Boot Loader to Load a Boot Application

The boot loader can be used to load the VxWorks boot application. You must build the binary image file and copy it to the USB flash drive under the name **/EFI/BOOT/bootapp.sys**. For more details, change to the directory **C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi** and refer to the file *README.txt*.

Once the boot application has been loaded and is running, it can then be used to load the VxWorks kernel image file.

The VxWorks kernel and the boot application both support warm and cold reboot.

```
BOOTX64.EFI +-> (vxWorks_romCompress.bin) -+-> (vxWorks) kernel ->+
        |   ^                               |                     |
        |   |                               V                     V
        |   + -----------(ctrl-x)----------+<-----(ctrl-x)--------+
        |                                                         |
        +<----------------------(cold)---------------------------+
```

Using the UEFI Boot Loader to Load the VxWorks Kernel

The boot loader can be used to load the VxWorks kernel image file. You must either build the binary image file or ELF file and copy it to the USB flash drive under the name **/EFI/BOOT/bootapp.sys**. For more details, change to the directory **C:\WindRiver\vxworks-7\pkgs_v2\boot\uefi** and refer to the file *README.txt*.

The VxWorks kernel only support cold reboot.

```
BOOTX64.EFI +-> (vxWorks.bin) ->+
        |                       |
        |                       V
        +<-------(cold)---------+

BOOTX64.EFI +-> (  vxWorks  ) ->+
        |                       |
        |                       V
        +<-------(cold)---------+
```

Parent topic: UEFI Boot Loader on page 31

6. BIOS Boot Loader

The BIOS boot loader can be used to load VxWorks.

The BIOS boot loader replaces the first stage in with the multi-stage boot application. Refer to Multi-Stage Boot Application on page 19 for details on how to build and deploy the multi-stage boot application. You must build the BIOS boot loader separately from the multi-stage boot application, and deploy them both on the target boot device.

For more information about the capabilities of the BIOS boot loader, navigate to the directory *installDir* **/vxworks-7/pkgs_v2/boot/bios** and refer to *README.txt*.

Building and Deploying the BIOS Boot Loader on page 36 The BIOS boot loader must be built before it is deployed on your boot device.
Operation of the BIOS Boot Loader on page 37 The BIOS boot loader loads various kernel types, and supports both warm and cold reboot.

6.1. Building and Deploying the BIOS Boot Loader

The BIOS boot loader must be built before it is deployed on your boot device.

Before You Begin

Before you begin, ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

Procedure

1. Change to the **bios** directory.

```
C:\WindRiver\workspace> cd ..\vxworks-7\pkgs_v2\boot\bios
C:\WindRiver\vxworks-7\pkgs_v2\boot\bios>
```

2. Build the bios boot loader.

```
C:\WindRiver\vxworks-7\pkgs_v2\boot\bios> make
```

3. Locate the bios boot loader executable file that you have built.

```
C:\WindRiver\vxworks-7\pkgs_v2\boot\bios> cd C:\WindRiver\vxworks-7\workspace
C:\WindRiver\vxworks-7\workspace> cd bios
C:\WindRiver\vxworks-7\workspace\bios> dir vxStage1Boot.bin

 Directory of C:\WindRiver\vxworks-7\workspace\bios

02/09/2018  15:09           23,179 vxStage1Boot.bin

C:\WindRiver\vxworks-7\workspace\bios>
```

4. Refer to Multi-Stage Boot Application on page 19 for instructions to deploy the multi-stage boot loader to the target boot device. You must substitute the BIOS boot loader file **vxStage1Boot.bin** that you have just built while following these steps.

Parent topic: BIOS Boot Loader on page 36

6.2. Operation of the BIOS Boot Loader

The BIOS boot loader loads various kernel types, and supports both warm and cold reboot.

The BIOS reads the boot device at sector zero and locates the VxWorks loader (vxld) there. The target executes vxld, it then loads and executes the BIOS boot loader, **vxStage1Boot.bin** (renamed **bootrom.sys**), which in turn loads the second stage boot application, **vxWorks_romCompress.bin** (renamed **bootapp.sys**). The boot application loads the VxWorks kernel image file as normal.

The BIOS boot loader supports both warm and cold boot for both the kernel and the boot application.

```
(vxStage1Boot.bin) +-> (vxWorks_romCompress.bin) -+-> (vxWorks) kernel ->+
                   |   ^                           |                     |
                   |   |                           V                     V
                   |   + -----------(ctrl-x)----------+<-----(ctrl-x)--------+
                   |                                                         |
                   +<----------------------(cold)----------------------------+
```

Parent topic: BIOS Boot Loader on page 36

4. VIRTUAL MACHINE BOOT WORKFLOWS

1. Supported Virtual Machine Workflows

VxWorks can be booted as a virtual machine under the following product technologies:

- Wind River Simics
- Helix Platform
- QEMU/KVM
- VMWare
- Virtualbox

The information in this chapter is presented as a starting point for booting VxWorks as a virtual machine, it is not a fully comprehensive guide. The third party product technologies evolve over time, and this is bound to alter the presented VxWorks workflow. Wind River recommends you consult with third party documentation before attempting to boot VxWorks as a virtual machine.

2. Creating VxWorks Projects for Virtual Machine

Before You Begin

Before you begin, ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

Procedure

1. Create a uniprocessor VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -bsp itl_generic -cpu NEHALEM -up -lp64 -debug
vsb_NEHALEM_64_up -S
```

> Note: Alternatively, you may create a symmetric multiprocessor VSB. Replace the parameter -up with the parameter -smp and adjust the VSB project name accordingly.

2. Build the project.

```
C:\WindRiver\workspace> cd vsb_NEHALEM_64_up
C:\WindRiver\workspace\vsb_NEHALEM_64_up> vxprj build
C:\WindRiver\workspace\vsb_NEHALEM_64_up> cd ..
```

3. Create a multi-stage boot application with symbolic debug information.

```
C:\WindRiver\workspace> vxprj create itl_generic itl_generic_nehalem_bootapp -profile PROFILE_B
OOTAPP -lp64 -debug -vsb vsb_NEHALEM_64_up
```

4. Build the project.

```
C:\WindRiver\workspace> cd itl_generic_nehalem_bootapp
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build set default_romCompress
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj vip component add INCLUDE_PC_CONSOLE
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build vxWorks_romCompress.bin
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> vxprj build vxStage1Boot.bin
C:\WindRiver\workspace\itl_generic_nehalem_bootapp> cd ..
```

Note: If you intend to run the boot application on Wind River Simics, omit the component **INCLUDE_PC_CONSOLE** from the project.

5. Create a VxWorks project.

```
C:\WindRiver\workspace> vxprj create itl_generic itl_generic_nehalem_kernel_up -profile PROFILE
_INTEL_GENERIC -lp64 -debug -vsb vsb_NEHALEM_64_up
```

Note: If you created a SMP VSB in step 1, you will created an SMP VIP at this step.

6. Build the project.

```
C:\WindRiver\workspace> cd itl_generic_nehalem_kernel_up
C:\WindRiver\workspace\itl_generic_nehalem_kernel_up> vxprj build
C:\WindRiver\workspace\itl_generic_nehalem_kernel_up> cd ..
```

3. VMWare Workflow

You boot VxWorks in VMWare player by first creating a bootable USB flash drive to load the VxWorks kernel into the virtual machine. The USB flash drive can be configured in one of two ways:

1. a VxWorks multi-stage boot application installed on the USB flash drive.
2. a multiboot *GRUB 2.00* boot loader installed on the USB flash drive.

   You will define the virtual machine to use the USB flash drive as its file system. The virtual machine will boot into the boot loader and you will then load the VxWorks kernel image either from the USB flash drive or the network.

Note: When you create the VIP for the kernel, you must not include the kernel component **INCLUDE_HPET_SYS_CLK**. VxWorks will not boot in VMWare with HPET comparitors active.

Creating a USB Flash Drive Boot Disk With Multi-Stage Boot Application on page 39
Creating a VxWorks VMWare Virtual Machine on page 40

3.1. Creating a USB Flash Drive Boot Disk With Multi-Stage Boot Application

Before You Begin

These steps assume:

1. You have identified a USB flash drive to use with VxWorks virtualisation on VMWare.
2. You have inserted your USB flash drive into the workstation.

3. You have created a built a multi-stage boot application project called itl_generic_NEHALEM_bootapp.
4. You are logged in as root.

Procedure

1. Zero out the VxWorks disk image.

```
% dd if=/dev/zero of=/directory/vxworks_disk.img bs=1M count=512
```

2. Setup the VxWorks disk image partition using the utility **parted**.

```
(parted) mktable msdos
(parted) mkpart
Partition type? primary/extended? primary
File system type? [ext2]? fat16
Start? 0
End? 2
Ignore/Cancel? Ignore
(parted) toggle 1 boot
(parted) quit
```

3. Copy the VxWorks multi-stage boot application to the VxWorks disk partition.

```
% dd if=installDir/vxworks-7/host/x86-linux2/bin/vxld.bin of=/directory/vxworks_disk.img bs=51
2 count=1 seek=0 conv=notrunc
% mdel -i /directory/vxworks_disk.img ::/bootrom.sys ::/bootapp.sys
% mcopy -i /directory/vxworks_disk.img installDir/workspace/itl_generic_NEHALEM_bootapp/default
_romCompress/vxStage1Boot.bin ::bootrom.sys
% mcopy -i /directory/vxworks_disk.img installDir/workspace/itl_generic_NEHALEM_bootapp/default
_romCompress/vxWorks_romCompress.bin ::bootapp.sys
```

Parent topic: VMWare Workflow on page 39

3.2. Creating a VxWorks VMWare Virtual Machine

Before You Begin

These steps require that:

1. your USB flash drive has either GRUB 2.0 or the VxWorks multi-stage boot application installed on it.
2. the USB flash drive is inserted into your workstation.
3. you are logged into your workstation as root.

Procedure

1. Create a new virtual machine in **VMWare Player**. Choose the following VMWare virtual machine options:
   - **I will install the operating system later**
   - **Guest Operating System: Other**
   - **Version: Other 64-bit**
2. Edit the virtual machine settings. Choose the following VMWare virtual machine options:

- **Memory: 2048 MB**
- **Processors: 2**
- **Network Adapter: Bridged**
- Delete the Sound Card
- Delete the default hard disk

- Add a new hard disk with the following options:
  - **IDE**
  - Use a physical disk and select the bootable USB flash drive
  - Use entire disk

3. Run the virtual machine in **VMWare Player**.

Parent topic: VMWare Workflow on page 39

4. Wind River Simics Workflow

The Simics script you use to boot VxWorks is called **x86-x58-ich10-vxworks-7.0-smp-32-multi**. You will use this Simics script to boot VxWorks kernel configurations both 32-bit and 64-bit, and both uniprocessor and symmetric multiprocessor. The default network IP addresses setup by the script are:

- *$ip_address* =10.10.0.30
- *$host_ip* =10.10.0.1
- *$gateway_ip* =10.10.0.1
- *$netmask* =ffffff00

Note: The Simics service node is configured with FTP server support.

Simics will boot VxWorks in a variety ways:

1. using a VxWorks loader boot disk image.
2. using a USB flash drive with a multiboot *GRUB 2.00* boot loader installed on it.

You must configure both your boot application project and VIP project with the **INCLUDE_PC_CONSOLE** component removed, because the boot application and the kernel will boot with serial console support.

Wind River recommends you thoroughly read the documentation you received with your Simics product before attempting this VxWorks virtualization workflow.

Configure Simics Script on page 41

4.1. Configure Simics Script

Before You Begin

These steps assume:

1. you have created a disk image containing the correct boot loader configured to use the serial console.
2. you have created a VIP configured to use the serial console.
3. your workstation has a TFTP server running.
4. the TFTP sever directory is **/tftpboot** and it has been setup to contain the VIP vxWorks image file.

Procedure

1. Start Simics by running **simics-eclipse**.
2. Click the **Start a New Simics Session** button (it looks like a butterfly).
3. Setup the **Simics Configuration to Launch** window:
    1. Click **New Project**.
    2. Accept the default values and click **Finish**.
    3. Click **Target system** and choose both **x86-x58-ich10** and **x86-x58-ich10-vxworks-7.0-smp-32-multi** from the drop-down menus.
    4. Click **Start as debug session**.
    5. Click **Advanced Options**.
    6. Select **Start reverse execution state collecting at launch**.
    7. In the **Before Loading Configuration** box, enter the following variables:
        - *$disk_image* =/directory/vxworks_disk.img
        - *$ftp_path* =/tftpboot
        - *$boot_file* =vxWorks
        - *$machine_count* =1
    8. Click **OK**.
    9. Click **OK**.
4. Press the **F8** key to resume.

Parent topic: Wind River Simics Workflow on page 41

5. Helix Platform Workflow

Refer to *Helix Platform Configuration and Build Guide: About Standard VxWorks 7 Guests*.

6. QEMU/KVM Workflow

You boot VxWorks in *QEMU/KVM* by first creating a bootable USB flash drive to load the VxWorks kernel into the virtual machine. The USB flash drive must be configured with a multiboot *GRUB 2.00* boot loader.

You will use the QEMU/KVM workflow to boot VxWorks kernel configurations of both 32-bit and 64-bit, and both uniprocessor and symmetric multiprocessor. If your VIP is a 64-bit VxWorks kernel, you must always give the QEMU machine more than 512 MB of memory. The 64-bit VxWorks kernel requires a heap size of 512MB in size, and so you must increase the overall memory size for the target to account for that. Use the *-m* QEMU option to increase the default memory size when running a 64-bit VxWorks kernel image on QEMU.

Booting VxWorks on QEMU Using GRUB on page 43
Booting VxWorks Directly on QEMU on page 43

6.1. Booting VxWorks on QEMU Using GRUB

Before You Begin

These steps assume you have a USB flash drive configured to boot the target into GRUB2.0. The steps refer to the USB flash drive as **/directory/vxworks_disk.img**.

Note: Versions of GRUB other than version 2.0 will not boot a 64-bit VxWorks image.

Procedure

1. Install the VxWorks guest operating system.

```
~/home/user$ qemu-system-x86_64 -machine q35 -enable-kvm -m 2048 -smp 2 --serial stdio -net ni
c -drive file=/directory/vxworks_disk.img,format=raw
```

2. Run the VxWorks guest operating system.

```
~/host/user$ qemu-kvm -machine q35 -m 2048 -smp 2 -serial stdio -net nic -drive file=/directory
/vxworks_disk.img,format=raw
```

Note: VxWorks target networking requires further configuration, but this is not covered by these steps.

Parent topic: QEMU/KVM Workflow on page 42

6.2. Booting VxWorks Directly on QEMU

Before You Begin

These steps require that:

1. You must use itl_generic BSP version **1.0.2.0** or later.
2. Your QEMU distribution includes the following commit:
   http://git.qemu.org/?p=qemu.git;a=commit;h=96a3d3927730c9445983f50ac0f55e0fce3851fa

Procedure

1. Install the VxWorks guest operating system.

```
~/home/user$ qemu-system-x86_64 -machine q35 -enable-kvm -m 2048 -smp 2 --serial stdio -net use
r -net nic
      -append "bootline:fs(0,0)host:vxWorks h=10.0.2.2 e=10.0.2.3 u=target o=gei0" -kernel
      /directory/vxworks_disk.img
```

2. Run the VxWorks guest operating system.

```
~/home/user$ qemu-kvm -machine q35 -m 2048 -smp 2 -serial stdio -net nic -kernel
      /directory/vxworks_disk.img
```


WNDRVR

Note: VxWorks target networking requires further configuration not covered by these steps.

Parent topic: QEMU/KVM Workflow on page 42

7. VirtualBox Workflow

You can boot VxWorks in a VirtualBox virtual machine by using a USB flash drive configured either with the VxWorks multi-stage boot application or the multiboot *GRUB 2.0* bootloader.

Converting the USB Flash Drive Image to a Virtual Box File on page 44
Creating a VxWorks VirtualBox Virtual Machine on page 44

7.1. Converting the USB Flash Drive Image to a Virtual Box File

Before You Begin

These steps assume that you have a USB flash drive, installed with either VxWorks multi-stage boot application or multiboot *GRUB 2.0* boot loader, inserted into your workstation.

Procedure

Convert the USB flash drive disk image to VirtualBox compatible image.

```
~/home/user$ VBoxManage convertfromraw /directory/vxworks_disk.img /directory/vxworks_virtualbox_d
isk.vmdk --format VMDK
~/home/user$ sudo VBoxManage internalcommands createrawvmdk -filename /directory/bootable_usb_flas
h_drv.vmdk -rawdisk /dev/sdc
```

Note: This step assumes the USB flash drive has been mounted by the workstation as **/dev/sdc**.

Parent topic: VirtualBox Workflow on page 44

7.2. Creating a VxWorks VirtualBox Virtual Machine

Before You Begin

These steps require you to have converted your USB flash drive image to a *VirtualBox* compatible file.

Procedure

1. Create a virtual machine as follows:
    1. **Name: x86_vxworks_virtualbox**
    2. **Type: Other**
    3. **Version: Other/Unknown (64-bit)**
    4. **Memory size: 2048 MB**

5. Hard drive: Use an existing virtual hard drive file **/directory/vxworks_virtualbox_disk.vmdk**

2. Configure the virtual machine settings as follows:
   1. **System →Motherboard →Enable IO APIC**
   2. **System →Processor →Processors: 2**
   3. **Display →Video →Video Memory: 32 MB**
   4. **Audio →Disable Audio**
   5. **Network →Adapter 1 →Enable Network Adapter**
   6. **Network →Adapter 1 →Attached to: Bridged Adapter**
   7. **Network →Adapter 1 →Name: vmnet1**
   8.

   Network →Advanced →Adapter Type: Intel Pro/1000 MT Desktop

   Note: **vmnet1** is for VxWorks networking and may require further configuration.

3. Enable HPET in the virtual machine.

```
~/home/user$ VBoxManage modifyvm x86_vxworks_virtualbox --hpet on
```

Note: The VxWorks kernel will not boot as a VirtualBox virtual machine if **INCLUDE_HPET_SYS_CLK** is included in the VIP.

Parent topic: VirtualBox Workflow on page 44

5. DEVICE SPECIFIC INFORMATION

1. VxBus Device Driver Details

Device drivers in the **itl_generic** BSP are implemented as part of the the VxWorks **VxBus** system. This section provides some brief notes on those drivers. For further details, refer to the entry for the driver in *API Reference: Kernel Core OS*.

vxbIaNs16550Sio on page 46
pcConsole on page 46
m6845Vga on page 46
vxbPciAhciSata on page 47
HPET (High Precision Event Timer) on page 47
vxbI8253Timer on page 47
Clock Rate Set Functions on page 48
i8253 Clock Frequency on page 48
vxbMc146818Rtc on page 48
vxbIntelTimestamp on page 48
vxbLoApicTimer on page 48
APIC/xAPIC Frequency on page 48
nullNvRam on page 49
vxbGei825xxEnd on page 49
USB Support on page 49

1.1. vxbIaNs16550Sio

Used for the serial ports. This driver does not work with targets that have

an E7520 chipset.

Parent topic: VxBus Device Driver Details on page 46

1.2. pcConsole

The **INCLUDE_PC_CONSOLE** component must be included in your boot application and VxWorks kernel projects to add PC console support.

```
~/home/user/WindRiver/workspace$ vxprj vip component add INCLUDE_PC_CONSOLE
```

Note: **DRV_KBD_USB** is used to enable the USB keyboard as a target input device. When added, **DRV_KBD_USB** will also pull in USB EHCI and XHCI host controller driver components. Some boards may not support these additional host controller drivers. Remove any driver components that are not supported by your board.

Parent topic: VxBus Device Driver Details on page 46

1.3. m6845Vga

Driver for the Motorola M6845 VGA controller. To use this controller, include the **INCLUDE_PC_CONSOLE** component in your project.

Parent topic: VxBus Device Driver Details on page 46

1.4. vxbPciAhciSata

The Vxbus Driver for the SATA hard disk controller. To use this driver, the component **PCI_DRV_STORAGE_AHCI** must be included in your project.

```
~/home/user/WindRiver/workspace$ vxprj vip component add PCI_DRV_STORAGE_AHCI
```

Parent topic: VxBus Device Driver Details on page 46

1.5. HPET (High Precision Event Timer)

This is the main counter of the **HPET**, which can be used for the VxWorks timestamp. For an explanation of the timestamp selection process between the TSC and HPET timestamps, see the `Timestamp Configuration' section.

Parent topic: VxBus Device Driver Details on page 46

1.6. vxbI8253Timer

This library contains a board-independent interface for manipulating the timer functions on Intel 8253 and compatible timer chip devices.

This driver is configured using settings in **20IaDrvCfg.cdf** for the **DRV_TIMER_I8253** component.

Wind River strongly recommends that you consult the documentation for the target hardware platform prior to updating any driver settings. In particular, you must know how the outputs of the timer channels are connected on the target hardware platform.

As an example of why it is important to consider how an 8253-compatible device is integrated into the system, consider how such devices were often implemented in legacy consumer desktop applications. The 8253-compatible chips usually contain three timers. Typically, all three timers are driven by a 14.31818 MHz crystal input from the system board, divided by 12, to yield a 1.19318 MHz input clock to the timers. The outputs from each timer channel were, and are, often connected as follows in desktop systems:

```
                      8253
                +---------------+
                |    Timer 2    |
from bit 0      |         output+------> to speaker circuitry
of port 61h ----+->gate         |
                |               |
1.19318 MHz ----+->clk 2        |
                |               |
                +---------------+
                |    Timer 1    |
+5 V            |         output+------> DRAM refresh
(logic 1)--+----+->gate         |
           |    |               |
1.19318 MHz ----+->clk 1        |
           |    |               |
           |    +---------------+
           |    |    Timer 0    |
           |    |         output+------> to IRQ0 (timer interrupt)
           +----+->gate         |
                |               |
1.19318 MHz ----+->clk 0        |
                |               |
                +---------------+
```

As indicated in the diagram, the output of timer channel 2 is connected to nothing other than the speaker. The output of timer 2 is not connected to the 8259 PIC or other type of interrupt controller.

The output from timer channel 1 is dedicated to providing DRAM refresh. As a result, this timer must not be manipulated once it is programmed appropriately for the system DRAM.

Because the output from timer channel 0 in the example above is connected to an interrupt controller and is not used as a time base for a system critical function (such as DRAM refresh), timer 0 is a good candidate for use as a programmable system or auxiliary clock device.

The example above is but one possible way 8253-compatible timer devices might be integrated into a target system. Some system boards may connect all timer channel outputs to an interrupt controller. Not every system will connect timer channel outputs to DRAM refresh or to a speaker. Again, you are encouraged to consult the target hardware documentation in order to understand the requirements for a particular system.

Parent topic: VxBus Device Driver Details on page 46

1.7. Clock Rate Set Functions

The macros **SYS_CLK_RATE_MIN**, **SYS_CLK_RATE_MAX**, **AUX_CLK_RATE_MIN**, and **AUX_CLK_RATE_MAX** must be defined to provide parameter checking for the sys[Aux]ClkRateSet( ) functions.

Parent topic: VxBus Device Driver Details on page 46

1.8. i8253 Clock Frequency

The **PIT_CLOCK** macro must also be defined to indicate the clock frequency of the i8253.

Parent topic: VxBus Device Driver Details on page 46

1.9. vxbMc146818Rtc

This is the driver for the real-time clock (based on the Motorola MC146818). It can be configured both as a real-time clock **DRV_TIMER_MC146818** and as an auxiliary clock **INCLUDE_MC146818_AUX_CLK**. The **DRV_TIMER_MC146818** component is added automatically when **INCLUDE_MC146818_AUX_CLK** component is added.

Parent topic: VxBus Device Driver Details on page 46

1.10. vxbIntelTimestamp

This is the driver for the Timestamp Counter (TSC) timestamp on Intel chipsets.

For an explanation of the timestamp selection process between the TSC and HPET timestamps, see the `Timestamp Configuration' section of this document.

Parent topic: VxBus Device Driver Details on page 46

1.11. vxbLoApicTimer

This library contains functions to manipulate the timer functions on the local APIC/xAPIC timer with a board-independent interface.

Parent topic: VxBus Device Driver Details on page 46

1.12. APIC/xAPIC Frequency

At initialization, the vxbLoApicTimer( ) timer driver calculates the clock frequency of the local APIC/xAPIC timer only when the **DRV_TIMER_LOAPIC_INSTANCE_0_CLK_FREQ** parameter is set to 0 (default). Otherwise, the specified value is applied as the

WNDRVR

clock frequency. If this value is set manually, it must match the actual frequency of the timer hardware to ensure accurate timekeeping. Platform-specific operating frequencies can only be obtained from the appropriate Intel proprietary documentation. This driver is configured using settings in **20laDrvCfg.cdf**.

This value can be set using Workbench or with the vxprj commmand line (VIP):

```
~/home/user/WindRiver/workspace$ vxprj vip parameter set DRV_TIMER_LOAPIC_INSTANCE_0_CLK_FREQ 1333
33333
```

Note: For clock speed determinism, disable **SpeedStep** in the BIOS.

Parent topic: VxBus Device Driver Details on page 46

1.13. nullNvRam

This library contains dummy non-volatile RAM manipulation functions for targets lacking non-volatile RAM. This library includes read and write functions that return ERROR.

The **NV_RAM_SIZE** macro must be configured as NONE for targets lacking non-volatile RAM.

For vxprj based images, the default value for **NV_RAM_SIZE** is set based on the **SYS_WARM_TYPE** parameter. **NV_RAM_SIZE** is set to NONE for the **SYS_WARM_BIOS** setting. For information on **SYS_WARM_START**configuration, see the section on warm boot devices.

Parent topic: VxBus Device Driver Details on page 46

1.14. vxbGei825xxEnd

This library supports the PCH-integrated network driver. This supports 10/100/1000 Mbps Ethernet through the onboard LAN PHY devices.

Parent topic: VxBus Device Driver Details on page 46

1.15. USB Support

This BSP supports USB attached devices.

Flash drives, hubs, keyboards, and mice have been tested with this BSP. At this time, not all USB hubs are supported. If you experience an issue with a USB hub, file a support request with Wind River support to see if there are any current patches for the USB stack.

Finally, when using flash drives, be sure to set the **BULK_MAX_DRIVES** configuration parameter to the maximum number of flash drives that can be inserted simultaneously (the default is 2). Set **BULK_NON_REMOVABLE_DISK** to TRUE (default is FALSE). Note that setting **BULK_NON_REMOVABLE_DISK** to FALSE can cause exceptions in the USB stack.

Parent topic: VxBus Device Driver Details on page 46

2. Interrupts

This BSP supports the Advanced Programmable Interrupt Controller (APIC/xAPIC), which consists of local APIC/xAPIC (on-chip) and I/O APIC/xAPIC (on the chipset).

Symmetric I/O Mode on page 50
mpApicData Structure on page 50
mpApicData Structure Configuration on page 50
Retrieving ACPI Generated Information for Use In a Static mpApicData Structure on page 51
Configuring the Target BIOS on page 51

2.1. Symmetric I/O Mode

One of the three interrupt modes defined by the MP specification is called **Symmetric I/O Mode**. In this mode, the APICs are fully functional, and interrupts are generated and delivered to the processors. Any interrupt can be delivered to any processor. This is the only multiprocessor interrupt mode. If the local APIC exists in the processor as indicated by the APIC feature flag in the CPUID and the I/O APIC in the chipset is available, this mode can be configured and used. The PIRQ[n] is directly handled by the I/O APIC in this mode.

Symmetric I/O Mode is the only interrupt mode available for the **itl_generic** BSP.

Parent topic: Interrupts on page 49

2.2. mpApicData Structure

You can configure the mpApic data table statically in this BSP. The static configuration table for the **mpApicData** structure allows for manual configuration of the **mpApic** driver. You must be careful when configuring APICs. Incorrect data causes catastrophic errors on the target, including the BSP failing to boot properly.

The data being manipulated is dependent on the number of CPUs involved, interrupt routing tables, and so forth; this is specific to the target platform. You can access this capability by adding the **INCLUDE_USR_BOOT_OP** component to your project.

For details on using a static mpApic data table, see the "MP APIC Configuration Using Command-Line Tools" section.

Parent topic: Interrupts on page 49

2.3. mpApicData Structure Configuration

Symmetric I/O Mode allows for three different approaches to creating the mpApicData structure:

1. **INCLUDE_MPTABLE_BOOT_OP**

   The structure is created using the BIOS MP table. Select this option in the VxWorks Image Project (VIP) as follows:

   ```
   ~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_MPTABLE_BOOT_OP
   ```

2. **INCLUDE_ACPI_BOOT_OP**

   The structure is created using the ACPI tables. This is the default VIP configuration, but can be selected as follows:

   ```
   ~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_ACPI_BOOT_OP
   ```

3. **INCLUDE_USR_BOOT_OP**

   The structure is created using a user-defined, static configuration.

   ```
   ~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USR_BOOT_OP
   ```

Parent topic: Interrupts on page 49

2.4. Retrieving ACPI Generated Information for Use In a Static mpApicData Structure

Before You Begin

These steps assume you have a multi-stage boot application project and a VxWorks Image Project (VIP) currently configured with **BUNDLE_STANDALONE_SHELL** and **INCLUDE_ACPI_BOOT_OP** and you need to switch the VIP to use **INCLUDE_USR_BOOT_OP** instead.

Procedure

1. Change to your VIP directory.
2. Add the component **INCLUDE_MPTABLE_SHOW**.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_MPTABLE_SHOW
```

3. Rebuild the VIP and reboot the target into VxWorks.
4. At the VxWorks kernel shell print out the contents of the running target's *mpApicData* structure:

```
-> vxMpTablePrint()
```

5. Locate the static structure in the BSP file **usrMpApicData.c**. This file is included by the Intel platform source file **usrItlCommon.c**.
6. Copy the output from vxMpTablePrint( ) on the target to the static structure in **usrMpApicData.c**.
7. Change to the multi-stage boot application project directory and enable **INCLUDE_USR_BOOT_OP**.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USR_BOOT_OP
```

> Note: The function sysStaticMpApicDataGet( ) in **usrItlCommon.c** copies the static *mpApicData* structure to the MP Data structure in target memory.

8. Rebuild the boot aplication project and deploy it to the target.
9. Change to the VIP directory and enable **INCLUDE_USR_BOOT_OP**.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USR_BOOT_OP
```

10. Reboot the target and VxWorks will use the statically configured *mpApicData* structure.

Parent topic: Interrupts on page 49

2.5. Configuring the Target BIOS

Before You Begin

These steps assume the specified options are available on your target BIOS. Not all BIOS options are the same, these steps may or may not be applicable to your target board.

Procedure

1. Boot the target board into the BIOS.
2. Set the BIOS options to **Optimal Defaults**.
3. Set the MPS configuration defaults to **MPS 1.4**.
4. Ensure the **Super I/O Configuration** (config SMC 17x Super I/O Chipset) sets **serial** and **LPT** devices to **DISABLED**.

Parent topic: Interrupts on page 49

3. AHCI, ATA and SCSI

Ensure the target BIOS does not have both AHCI and ATA (legacy IDE) is enabled at the same time. If they are, neither the VxWorks multi-stage boot application or the VxWorks Image Project (VIP) will complete AHCI driver initialization and so will not boot.

SCSI configuration is not applicable for the itl_generic BSP unless a PCIe or PCI SCSI card is plugged into the target board. For this to work, the SCSI chip must be supported by Wind River and have a VxWorks driver. For more information, contact Wind River support.

4. Serial Configuration

The default serial configuration is:

- Baud Rate: 9600
- Data: 8 bit
- Parity: None
- Stop: 1 bit
- Flow control: None

5. Network Configuration

The itl_generic BSP supports the following network driver components:

1. **INCLUDE_FEI8255X_VXB_END**
2. **INCLUDE_GEI825XX_VXB_END**
3. **INCLUDE_RTL8169_VXB_END**
4. **DRV_VXBEND_TEI82598**

No modification to the BSP is required, and adding these driver components to the project also includes the required dependencies.

6. HPET Configuration

The itl_generic BSP supports the use of the HPET as a system clock, auxiliary clock or as the timestamp clock. When using the HPET for a system or auxiliary clock, the VxWorks kernel suffers from increased latency compared to the use of the local APIC timer. This is due to the HPET register access time of up to 2 microseconds.

The BSP supports 3 HPET timers using MSI interrupts. A particular platform may have additional HPET timers, or have timers that do not support MSI interrupts.

The components that configure the HPET are:

- **INCLUDE_HPET_SYS_CLK**
  Use the HPET timer as the system clock.
- **INCLUDE_HPET_AUX_CLK**
  Use the HPET timer as the auxiliary clock.
- **INCLUDE_HPET_TIMESTAMP**
  Use the HPET timer as a timestamp clock.

Each HPET timer is individually configured by modifying the following parameters with vxprj or Workbench:

- **DRV_TIMER_IA_HPET_INSTANCE_n_MODE**
- **DRV_TIMER_IA_HPET_INSTANCE_n_MIN_FREQ**
- **DRV_TIMER_IA_HPET_INSTANCE_n_MAX_FREQ**
- **DRV_TIMER_IA_HPET_INSTANCE_n_INT_TYPE**
- **DRV_TIMER_IA_HPET_INSTANCE_n_INT_IOAPIC_PIN**
- **DRV_TIMER_IA_HPET_INSTANCE_n_INT_VEC**
- **DRV_TIMER_IA_HPET_INSTANCE_n_TRIGGER**
- **DRV_TIMER_IA_HPET_INSTANCE_n_POLARITY**
- **DRV_TIMER_IA_HPET_INSTANCE_n_PERIODIC_SYNC**

To interrogate the target board HPET capabilities, include the component **DRV_TIMER_IA_HPET_BUS**. This includes the function vxbHpetShow( ), which must be called from the VxWorks kernel shell to report the target's HPET capabilities.

Each HPET parameter details, as well as examples of FDT formatting to add support for additional HPET timers, can be found in **vxbIaHpetBus.c**.

For an explanation of the timestamp selection process between the TSC and HPET timestamps, refer to Timestamp Driver on page 53.

7. Timestamp Driver

Timestamp support is incorporated into the VxWorks Image Project (VIP) by adding the **INCLUDE_TIMESTAMP** component. This component also includes either the **INCLUDE_TSC_TIMESTAMP** or **INCLUDE_HPET_TIMESTAMP** components, depending on the features of the hardware target. If an invariant (constant rate) TSC is available, the TSC timestamp is used. If an invariant (constant rate) TSC is unavailable, the HPET timestamp is used. If both invariant TSC and HPET are unavailable, a timer is selected from the remaining timers in the system. This is usually the local APIC timer.

Note: The local APIC timer must not be used as the timestamp clock in a symmetric multiprocessor (SMP) enabled VIP. Each CPU has its own local APIC timer and they are not synchronised in an SMP system architecture.

If you require 64-bit timestamps, you must not use the local APIC timer because it does not support 64-bit timestamps. You can enable 64-bit timestamp support by adding the **INCLUDE_TIMESTAMP64** component. The 64-bit capable HPET main counter can operate in 32-bit mode even when it is configured for 64-bit timestamps if the HPET's comparators are configured for 32-bit operation. This would be the case if the **INCLUDE_HPET_SYS_CLK** or **INCLUDE_HPET_AUX_CLK** component is included when building your image.

Including the TSC Timestamp Driver on page 54

WNDRVR

Including the HPET Timestamp Driver on page 54

7.1. Including the TSC Timestamp Driver

Before You Begin

These steps assume you have already created a VxWorks Image Project (VIP). Refer to Creating a VxWorks Image Project on page 13.

About This Task

Procedure

1. Change to the VIP directory.
2. Add the timestamp component to the VIP.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_TIMESTAMP
```

3. Set the VIP to use the TSC component.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_TSC_TIMESTAMP
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component remove INCLUDE_HPET_TIMESTAMP
```

4. **Optional:** If you need to use a variant (non-constant rate) TSC as a timestamp, the component parameter **DRV_TIMER_IA_TIMESTAMP_INSTANCE_n_INVARIANT_ONLY** must be adjusted from vxprj or Workbench. This parameter must be changed from its default value of 1, to 0.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set DRV_TIMER_IA_TIMESTAMP_INSTANCE_
0_INVARIANT_ONLY 0
```

> Note: This step only works for uniprocessor VxWorks kernels. This parameter is ignored for symmetric multiprocessor (SMP) kernels. Only an invariant TSC is allowed as a timestamp counter in SMP.

5. Include the component **INCLUDE_TIMER_SYS_SHOW** so that you can use vxbTimestampShow( ) from the kernel shell prompt to display the selected VxWorks timestamp configuration.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_TIMER_SYS_SHOW
```

Parent topic: Timestamp Driver on page 53

7.2. Including the HPET Timestamp Driver

Before You Begin

These steps assume you have already created a VxWorks Image Project (VIP). Refer to Creating a VxWorks Image Project on page 13.

Procedure

1. Change to the VIP directory.
2. Add the timestamp component to the VIP.

WNDRVR

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_TIMESTAMP
```

3. Set the VIP to use the HPET component.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_HPET_TIMESTAMP
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component remove INCLUDE_TSC_TIMESTAMP
```

4. Include the component **INCLUDE_TIMER_SYS_SHOW** so that you can use vxbTimestampShow( ) from the kernel shell prompt to display the selected VxWorks timestamp configuration.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_TIMER_SYS_SHOW
```

Parent topic: Timestamp Driver on page 53

8. Hyper-Threading Configuration

If your target board CPU supports hyper-threading, then after you enable hyper-threading in the BIOS you can add the **INCLUDE_SMT_ENABLE** component to enable hyper-threading in VxWorks. Do not enable this VxWorks component if the CPU does not support hyper-threading, or it is disabled in the BIOS. Refer to the *VxWorks 7 Intel Architecture Guide:About the Supported Intel Architectures* for more information about hyper-threading support in VxWorks.

For example, on the Intel **Walnut Canyon BRB** board, you enable hyperthreading in the BIOS as follows:

```
Advanced
  → CPU Configuration
    → SMT
      → Enabled
```

Controlling Hyper-Threading and Load Balancing in the VxWorks Kernel on page 55

8.1. Controlling Hyper-Threading and Load Balancing in the VxWorks Kernel

Before You Begin

These steps assume you have created a VxWorks Image Project (VIP). Refer to Creating a VxWorks Image Project on page 13.

Procedure

1. Change to the VIP directory.
2. **Optional:** To disable hyper-threading in VxWorks, remove the component **INCLUDE_SMT_ENABLE**.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component remove INCLUDE_SMT_ENABLE
```

Note: Hyper-threading support is added to the VIP by default.

3. **Optional:** To enable hyper-threading in VxWorks, add the component **INCLUDE_SMT_ENABLE**.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_SMT_ENABLE
```

4. **Optional:** To enable symmetric multiprocessing (SMP) load balancing with SMT/HT, include the component **INCLUDE_SMP_SCHED_SMT_POLICY**.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_SMP_SCHED_SMT_POLICY
```

Note: The component **INCLUDE_SMT_ENABLE** will also be included automatically.

Parent topic: Hyper-Threading Configuration on page 55

9. Symmetric Multiprocessing Application Processor (AP) Startup Time Period

You use two VxWorks Image Project (VIP) parameters to determine how long the system waits before it expects the CPUs in a multi-core processor to have booted. The system runs a test, and waits the defined period between tests.

1. **SYS_AP_TIMEOUT** is specified in microseconds.
2. **SYS_AP_LOOP_COUNT** is multiplied by **SYS_AP_TIMEOUT** to give the total waiting time for an Application Processor (AP) before the system gives up and moves on to the next AP.

   Set these parameters in the VIP project. For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set SYS_AP_LOOP_COUNT 200000
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set SYS_AP_TIMEOUT 10
```

10. SDHC/MMC Configuration

Secure Digital/Multimedia Card (SD/MMC) support is included in this BSP. You must include both the SD/MMC layer, which is not included by default, and the appropriate SD/MMC driver layers as well.

The SD/MMC layer must be included into the VxWorks Source Build (VSB) project, while the driver layers must be included into the VxWorks Image Project (VIP).

Adding the SD/MMC Components on page 56

10.1. Adding the SD/MMC Components

Before You Begin

These steps assume you have created both a VxWorks Source Build (VSB) and a VxWorks Image Project (VIP). Refer to Creating a Symmetric Multiprocessor VxWorks Source Build on page 7 and Creating a VxWorks Image Project on page 13.

Procedure

1. Change to the VIP directory.
2. Add the driver component.

WNDRVR

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_PCI_SDHC_CTRL
```

3. **Optional:** Add the MMC/eMMC driver component.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_MMCSTORAGE_CARD
```

4. **Optional:** Add the SD driver component.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_SDSTORAGE_CARD
```

Parent topic: SDHC/MMC Configuration on page 56

11. Adding CAN-PCIe/200 (SJA1000 Controller) Devices

Before You Begin

These steps assume you have created both a VxWorks Source Build (VSB) and a VxWorks Image Project (VIP). Refer to Creating a Symmetric Multiprocessor VxWorks Source Build on page 7 and Creating a VxWorks Image Project on page 13.

Procedure

1. Change to the VSB directory.
2. Add the CAN layer.

```
~/home/user/WindRiver/workspace/myVSB$ vxprj vsb add CAN_CTLR
```

3. Change to the VIP directory.
4. Add the SJA1000 component.

```
~/home/user/WindRiver/workspace/myVSB$ vxprj vip component add INCLUDE_PCI_SJA1000
```

12. Configuring Warm Boot Devices

Before You Begin

These steps assume you have created a VxWorks Image Project (VIP). Refer to Creating a VxWorks Image Project on page 13.

Procedure

1. Change to the VIP directory.
2. Select the appropriate warm boot device component.

Option	Description
BIOS Warm Boot Device	`~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_SYS_WARM_BIOS`
USB Warm Boot Device	`~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_SYS_WARM_USB`
AHCI Warm Boot Device	`~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_SYS_WARM_AHCI`

3. Set the *BOOTROM_DIR* parameter.

Option	Description
BIOS Warm Boot Device	`~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set BOOTROM_DIR`
USB Warm Boot Device (on Linux host)	`~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set BOOTROM_DIR '"/bd0"'`
AHCI Warm Boot Device (on Linux host)	`~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set BOOTROM_DIR '"/ata0:1"'`

13. pcConsole Configuration

The pcConsole configuration sets how VxWorks displays the target console on the target video display.

When a VxWorks image project (VIP) is built with the profile **PROFILE_INTEL_GENERIC** and based on the **itl_generic** BSP, it will include the component **INCLUDE_PC_CONSOLE**. Workbench assumes that you want the target console output to appear on the target video output rather than one of the target serial ports.

Note: You can switch the target console to use a serial port instead of the video output by removing the VIP component **INCLUDE_PC_CONSOLE**.

When added, the **INCLUDE_PC_CONSOLE** component brings in an additional PC console video driver component to the VIP. Two video driver components are available:

DRV_VGA_M6845(default)

Morotola 6845 VGA driver

DRV_CONSOLE_EFI

UEFI bitmapped console

Choosing the correct video driver component for your target will depend on the target's BIOS configuration. If you include the wrong video driver component, the VxWorks kernel will not boot successfully on the target, and you must switch to use the other video driver component.

There are two approaches to understanding how the target BIOS is configured:

1. Explore the BIOS settings directly on your target.
2. Boot the target into the Wind River EFI boot loader and observe whether VGA text mode is available or not.

Choosing the Video Component Based on BIOS Setting on page 59 The target BIOS boot settings tell you how to configure your VxWorks kernel.
Choosing the Video Component Based on the EFI Bootloader on page 60 The EFI bootloader output messages tell you how to configure your VxWorks kernel.
Secure EFI Bootloader Video Component on page 61 If you boot your target using the Wind River secure EFI bootloader, you must configure the VxWorks image project (VIP) with an EFI video driver component.

13.1. Choosing the Video Component Based on BIOS Setting

The target BIOS boot settings tell you how to configure your VxWorks kernel.

About This Task

It is important that you understand how the BIOS video boot settings are arranged so that you will configure your VxWorks image project (VIP) correctly. An incorrectly configured kernel will not boot properly.

Before You Begin

These steps assume you have powered on your target and entered the BIOS configuration. This process is target dependent. For some targets, you must press the delete key on the USB keyboard at boot time. Other targets require you to press the F12 key. Refer to the instructions on the target video output at boot time for details on how to enter your target's BIOS.

Procedure

1. Locate the BIOS boot settings.

   BIOS boot setting location is highly BIOS specific. For example:
   - On a DELL Latitude E6540, locate this setting under **General**, **Advanced Boot Options**, **Enable Legacy Option ROMs**.
   - On the American Megatrends BIOS 2.17.1246, released on 2016, the location is **Boot**, **CSM parameters**.

2. Determine whether or not legacy mode/CSM video mode is supported in the BIOS settings.
   - On a DELL Latitude E6540:

Option	Description
Enable Legacy Option ROMSs option checked	legacy mode/CSM video mode is supported
Enable Legacy Option ROMs option unchecked	legacy mode/CSM video mode is NOT supported

WNDRVR

- On the American Megatrends BIOS 2.17.1246:

Option	Description
Launch CSM and Launch Video OpRom Policy enabled	legacy mode/CSM video mode is supported
Launch CSM and Launch Video OpRom Policy disabled, UEFI only	legacy mode/CSM video mode is NOT supported

3. If legacy mode/CSM video mode is NOT supported, then you must change the VIP video component to **DRV_CONSOLE_EFI**:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add DRV_CONSOLE_EFI
```

Note: If you change the BIOS settings to enable legacy mode/CSM video mode, then you must revert the VIP back to the default video driver or else VxWorks will not boot:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add DRV_VGA_M6845
```

Parent topic: pcConsole Configuration on page 58

13.2. Choosing the Video Component Based on the EFI Bootloader

The EFI bootloader output messages tell you how to configure your VxWorks kernel.

About This Task

It is important that you understand how the BIOS video boot settings are arranged so that you will configure your VxWorks image project (VIP) correctly. An incorrectly configured kernel will not boot properly.

Before You Begin

These steps assume you have built and deployed the Wind River EFI bootloader in either signed or unsigned configuration. Instructions for building the unsigned configuration are located in the *UEFI readme*. Locate this file at *installDir* **/vxworks-7/pkgs_v2/boot/uefi-***w.x.y.z*. The signed bootloader configuration is documented in the VxWorks 7 Security Guide

Procedure

1. Power on the target and watch the EFI bootloader text.
2. Determine whether or not CSM video is supported.
   1. If the EFI bootloader reports VGA text mode support is available, this means CSM video is supported.

```
......
Total memory: xxxxxx

VGA text mode support available.

Booting VxWorks.
```


WNDRVR

```
Press a key to abort: 1
Jumping to VxWorks image at 0x4080x0...
```

2. If the EFI bootloader reports CSM video is not available, this means CSM video is NOT supported:

```
......
Total memory: xxxxxx

CSM video not available.

Booting VxWorks.
Press a key to abort: 1
Jumping to VxWorks image at 0x4080x0...
```

Note: The EFI bootloader reports the configuration of the target BIOS. To change the configuration, you must enter the target BIOS and modify the boot settings. Refer to Choosing the Video Component Based on BIOS Setting on page 59 for details.

3. If legacy mode/CSM video mode is NOT supported, then you must change the VIP video component to **DRV_CONSOLE_EFI**:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add DRV_CONSOLE_EFI
```

Note: If you change the BIOS settings to enable legacy mode/CSM video mode, then you must revert the VIP back to the default video driver or else VxWorks will not boot:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add DRV_
VGA_M6845
```

Parent topic: pcConsole Configuration on page 58

13.3. Secure EFI Bootloader Video Component

If you boot your target using the Wind River secure EFI bootloader, you must configure the VxWorks image project (VIP) with an EFI video driver component.

If you intend the boot the target into VxWorks securely using the EFI bootloader, the target BIOS settings will already need to be set so that legacy mode/CSM video mode is unsupported. When it comes time to configure the secure VIP, you must therefore add the component **DRV_CONSOLE_EFI**:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add DRV_CONSOLE_EFI
```

Parent topic: pcConsole Configuration on page 58

14. Low Power Subsystem (LPSS) Configuration

With LPSS feature, the LPSS devices can be placed into low power mode when they are idle. This is also called serial I/O. By default VxWorks always places the devices in full power mode.

About This Task

VxWorks 7 supports LPSS for UART and I2C devices that are located on the PCI system. LPSS is not supported for the same devices that are located on the ACPI system in this version of VxWorks 7.

Procedure

1. Add the component **DRV_INTEL_LPSS_PCI_UART** to your VIP to use LPSS UART. You may also include the PCI NS16550 UART component **DRV_SIO_PCI_NS16550** to your VIP as required.
2. Add the component **DRV_INTEL_LPSS_PCI_I2C** to your VIP to use LPSS I2C.

   Access to the I2C controller is required by certain devices such as EEPROMs or temperature sensors. Intel LPSS PCI I2C has an internal DesignWare I2C controller chip, so this driver has dependencies on the component **DRV_DW_I2C_COMMON** which holds the common part of DesignWare I2C controller chip.

   Note: LPSS I2C controller is capable of acting as a master only, slave mode is not supported. In this first release of the driver, very limited number of I2C slave devices have been tested. More I2C slave devices will be tested in future releases.

   1. To detect the I2C slave devices, the driver uses the information of slave devices from DTS.

   Each Intel LPSS PCI I2C controller has a unique PCI device id and vendor id, such as **0x8086:0xA160/0x8086:0xA161**, so the device could be probed by PCI system by checking the PCI device id and vendor id. This is not the case for slave devices (such as EEPROM/sensor) that reside on the I2C bus. So to detect the I2C slave devices, the driver uses the information of slave devices from DTS.

   Below is an example of DTS with information of slave device:

```
i2c_0_21_1 {
      #address-cells = <1>;
      #size-cells = <0>;
      eeprom@50{
               compatible = "at24,at24c512";
               reg = <0x50>;
               };
            };
```

   The I2C controller name in DTS should be of the format: i2c_*busId*_*devId*_*funcId*

   With the PCI information *busId*, *devId* and *funcId*, the I2C controller driver can match the exact I2C node in DTS and then parse the slave devices that belong to I2C controller. *busId*, *devId* and *funcId* of the I2C controller can vary from board to board. The user must check the hardware information of the target and update the **itl_generic.dts** accurately.

   Another easy way of obtaining *busId*, *devId* and *funcId* of the I2C controller is to include the component **DRV_INTEL_LPSS_PCI_I2C** into the VIP, rebuild the VIP and then use the PCI interface APIs vxbPciCtrlShow( )/ vxbPciTopoShow( ) to get *busId*, *devId* and *funcId* of the I2C controller when VxWorks is running. Then add the information to **itl_generic.dts** and rebuild VxWorks and reload the image.

   In addition to listing the information of slave device, some properties of I2C controller (like **def-bus-frequency** and **polled-mode**) can also be specified in the DTS.
   - **def-bus-frequency** - working clock frequency of I2C bus in Hz, if not specified **100000(Hz)** is used.
   - **polled-mode** - specifies work mode, 1 means polled-mode, 0 means interrupt mode. Interrupt mode is used if this property is not specified or defined with invalid values (only 0 and 1 are valid).

WNDRVR

For example:

```
i2c_0_21_1 {
      #address-cells = <1>;
      #size-cells = <0>;
      def-bus-frequency = <400000>;
      polled-mode=<1>;

      eeprom@50{
               compatible = "at24,at24c512";
               reg = <0x50>;
               };
           };
```

6. DEVICE CONFIGURATION INFORMATION

1. About Device Configuration on the itl_generic BSP

When working with an Intel-based system, you may need information specific to the itl_generic BSP regarding device enumeration and configuration with the flattened device tree (FDT) and ACPI.

The itl_generic BSP supports a variety of 32- and 64-bit Intel Architecture processors for VxWorks 7. It uses ACPI and FDT to facilitate device enumeration and configuration with VxBus.

The itl_generic BSP and its corresponding **ITL_COMMON** processor support library (PSL) support device enumeration using the VxBus drivers DRV_BUS_ACPI_ROOT and DRV_BUS_FDT_ROOT. Kernel image components for both drivers are included by default in boot application (**bootApp**) and VxWorks image project (VIP) images.

The DRV_BUS_ACPI_ROOT driver is used to dynamically discover hardware devices based on the ACPI data provided by the BIOS. This component is optional and you can remove it as needed.

The DRV_BUS_FDT_ROOT driver is used to statically declare hardware devices in the **itl_generic.dts** file in the BSP. You cannot remove this component from the BSP.

The itl_generic BSP requires the DRV_INTEL_NEXUS component. This component is used to override the default VxBus root nexus.

All drivers in the **ITL_COMMON** PSL are children of the FDT bus. Some drivers can also be children of the ACPI bus. For example, the serial port driver can be discovered by the ACPI bus or statically declared by the FDT bus. However, the 8253 timer driver can only be declared by the FDT bus.

Both the FDT and ACPI buses can discover a root PCI device. The architecture-independent PCI library then enumerates all PCI devices on the target hardware and attaches their respective drivers, if they are configured into your VIP. PCI drivers are independent of ACPI and FDT and require no special considerations when using this BSP.

Any driver that has user-customizable settings is configured through options in the **itl_generic.dts** file within the itl_generic BSP. These configuration options are used whether the device is discovered by ACPI or declared as a device entry in the **itl_generic.dts** file.

Example Static Device Declaration on page 64 Devices are declared statically in the BSP file **itl_generic.dts**.

1.1. Example Static Device Declaration

Devices are declared statically in the BSP file **itl_generic.dts**.

A statically declared example device is the board serial UART.

```
serial0: PNP0501 {
  io = <DRV_SIO_IA_NS16550_INSTANCE_0_BASE_ADDR 0x8>;
  interrupts = <DRV_SIO_IA_NS16550_INSTANCE_0_INT_VEC
            DRV_SIO_IA_NS16550_INSTANCE_0_INT_TYPE
            DRV_SIO_IA_NS16550_INSTANCE_0_INT_IOAPIC_PIN
            DRV_SIO_IA_NS16550_INSTANCE_0_TRIGGER
            DRV_SIO_IA_NS16550_INSTANCE_0_POLARITY>;
};
```

The I/O and interrupt settings for this device must be configured as VIP component parameters. This allows these parameters to be set from vxprj or Workbench. This approach to configuration is not an essential requirement, but makes configuration easier and more convenient.

WNDRVR

The default values in the **itl_generic.dts** file are selected to work on a wide variety of targets, so customization may be required.

Parent topic: About Device Configuration on the itl_generic BSP on page 64

2. Device Enumeration and Initialization

Refer to this topic for an explanation of the device enumeration and initialization process that can be helpful to you while developing with the itl_generic BSP.

The VxBus library creates an initial top-level device node that is responsible for creating all other device nodes. This node is provided by the DRV_INTEL_NEXUS driver and its associated attach function, intelNexusAttach( ).

First, the intelNexusAttach( ) function dynamically creates and attaches IoApic and LoApic interrupt controllers for each device present on the target hardware.

Next, the intelNexusAttach( ) function creates an ACPI bus node. If the DRV_BUS_ACPI_ROOT driver component is present in your VxWorks image project (VIP), the attach function for the driver executes. If the component is not included, ACPI devices will not be discovered during device enumeration.

The attach function for the DRV_BUS_ACPI_ROOT driver, acpiBusAttach( ), calls various ACPI library functions and creates a VxBus node for each valid ACPI device. The VxBus node is named from the hardware ID (HID) name provided by the ACPI device descriptor.

In order for a driver to attach to a device discovered by ACPI, the driver must set its **VXB_DRV** name to the matching HID found by ACPI and its class must be set to **VXB_BUSID_ACPI**.

For example, if the HID for a standard 16550 serial port is PNP0501. The driver definition is as follows:

```
VXB_DRV iaAcpiNs16550DrvRegistration =
    {
    { NULL },                    /* vxbNode */
    "PNP0501",                   /* Name */
    "iaNs16550 serial driver",   /* Description */
    VXB_BUSID_ACPI,              /* Class */
    0,                           /* Flags */
    0,                           /* Reference count */
    iaNs16550MethodList          /* Method table */
    };
```

After all devices are enumerated by the DRV_BUS_ACPI_ROOT driver's attach function, the intelNexusAttach( ) function creates an FDT bus node. The attach function for the driver component, fdtBusAttach( ), creates a VxBus device node for each device declared in the flattened device tree (FDT) file for the BSP.

If the device being declared in the FDT file can also be discovered by the ACPI bus, the device must use the matching HID as the node name.

For example, if the HID for a standard 16550 serial port is PNP0501, the driver and FDT node declaration is as follows.

In **serialDriver.c**:

```
VXB_DRV iaFdtNs16550DrvRegistration =
    {
    { NULL },                    /* vxbNode */
    "PNP0501",                   /* Name */
    "iaNs16550 serial driver",   /* Description */
```


WNDRVR

```
    VXB_BUSID_FDT,               /* Class */
    0,                           /* Flags */
    0,                           /* Reference count */
    iaNs16550MethodList          /* Method table */
    };
```

In **itl_generic.dts**:

```
serial0: PNP0501{
    io = <DRV_SIO_IA_NS16550_INSTANCE_0_BASE_ADDR 0x8>;
    interrupts = <DRV_SIO_IA_NS16550_INSTANCE_0_INT_IOAPIC_PIN
                  DRV_SIO_IA_NS16550_INSTANCE_0_INT_TYPE
                  DRV_SIO_IA_NS16550_INSTANCE_0_INT_VEC
                  DRV_SIO_IA_NS16550_INSTANCE_0_TRIGGER
                  DRV_SIO_IA_NS16550_INSTANCE_0_POLARITY>;
    };
```

In the following example, a VGA device is not discovered by ACPI and the **VXB_DRV** name field is set to **vxbM6845Vga**. The driver and FDT node declaration is as follows:

In **vgaDriver.c**:

```
VXB_DRV m6845vxbDevRegistration =
    {
    { NULL },                 /* vxbNode */
    "vxbM6845Vga",            /* pVxbName */
    M6845_DESC,               /* pVxbDesc */
    VXB_BUSID_FDT,            /* vxbClass */
    0,                        /* vxbFlags */
    0,                        /* vxbRefCnt */
    vxbM6845Vga_methods       /* pVxbMethods */
    };
```

In **itl_generic.dts**:

```
vxbM6845Vga{
    io = <DRV_VGA_M6845_INSTANCE_0_REG_BASE_ADRS 0x2>;
    reg = <DRV_VGA_M6845_INSTANCE_0_MEM_BASE_ADRS 0x1000>;
    };
```

In the above examples, the serial driver has two driver definitions (**VXB_DRV**), one with **vxbClass** set to **VXB_BUSID_FDT** and the other set to **VXB_BUSID_ACPI**. However, because the VGA device is a child of only the FDT bus, it has a single driver definition with the class set to **VXB_BUSID_FDT**.

3. VxWorks Image Project Device Configuration

The itl_generic BSP includes certain kernel configuration components by default. Depending on your needs, you may chose to customize the configuration.

When creating a VxWorks image project (VIP) based on the itl_generic BSP, the DRV_BUS_ACPI_ROOT and DRV_BUS_FDT_ROOT kernel configuration components are included by default. Using this configuration, the DRV_BUS_ACPI_ROOT driver discovers and enumerates all devices reported by ACPI on the target hardware. Any remaining devices declared in the flattened device tree (FDT)

WNDRVR

file are then enumerated. If you remove DRV_BUS_ACPI_ROOT from your VIP, the only devices that are enumerated are those that are declared in the FDT file in your BSP.

The default device configurations are those most commonly required for Intel hardware. You may need to customize the configuration to suit your target platform.

Adding an ns16550 Serial Port to a VIP on page 67 Add support for a serial port to your itl_generic-based project by configuring your VxWorks image project (VIP).

3.1. Adding an ns16550 Serial Port to a VIP

Add support for a serial port to your itl_generic-based project by configuring your VxWorks image project (VIP).

Before You Begin

These steps assume you have already created a VIP. For details, refer to Creating a VxWorks Image Project on page 13.

Procedure

1. Add the serial driver component.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_SIO_IA_NS16550
```

2. Add an instance of the device.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_SIO_IA_NS16550_INSTANCE_0
```

   Adding an instance of the device also adds the following parameters to your VIP:

   - DRV_SIO_IA_NS16550_INSTANCE_0_INT_TYPE
   - DRV_SIO_IA_NS16550_INSTANCE_0_INT_VEC
   - DRV_SIO_IA_NS16550_INSTANCE_0_INT_IOAPIC_PIN
   - DRV_SIO_IA_NS16550_INSTANCE_0_TRIGGER
   - DRV_SIO_IA_NS16550_INSTANCE_0_POLARITY
   - DRV_SIO_IA_NS16550_INSTANCE_REG_INTERVAL
   - DRV_SIO_IA_NS16550_INSTANCE_FIFO_LEN
   - DRV_SIO_IA_NS16550_INSTANCE_CLK_FREQ
   - DRV_SIO_IA_NS16550_INSTANCE_BAUD_RATE

3. Modify parameters as needed:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set DRV_SIO_IA_NS16550_INSTANCE_0_IN
T_IOAPIC_PIN 3
```

4. Add and configure additional serial port instances as needed.

   Additional instances serial port instances can be added and configured in the same manner.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_SIO_IA_NS16550_INSTANCE_1
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set DRV_SIO_IA_NS16550_INSTANCE_1_IN
T_IOAPIC_PIN 5
```


WNDRVR

In this example, DRV_SIO_IA_NS16550_INSTANCE_0 through DRV_SIO_IA_NS16550_INSTANCE_3 have already been declared as component description file (CDF) components. If more than 4 serial devices are required, you must add additional entries to the flattened device tree (FDT) and CDF files.

Each driver can have different options and each driver's source file contains documentation for each available option.

Parent topic: VxWorks Image Project Device Configuration on page 66

4. Device Configuration With FDT

The flattened device tree (FDT) is provided as part of a BSP and is used to provide configuration information for the devices in your system.

The **itl_generic.dts** file is a text file that is compiled into a binary object and linked with a VIP at build time. The file contains two main sections, the first contains device nodes that declare a device's name and resources. The second section declares device configuration.

Device nodes are declared in the following format:

```
serial0: PNP0501{
io = <DRV_SIO_IA_NS16550_INSTANCE_0_BASE_ADDR 0x8>;
interrupts = <DRV_SIO_IA_NS16550_INSTANCE_0_INT_VEC
    DRV_SIO_IA_NS16550_INSTANCE_0_INT_TYPE
    DRV_SIO_IA_NS16550_INSTANCE_0_INT_IOAPIC_PIN
    DRV_SIO_IA_NS16550_INSTANCE_0_TRIGGER
    DRV_SIO_IA_NS16550_INSTANCE_0_POLARITY>;
};
```

In the above example, component description file (CDF) parameters are used for IO and interrupt settings. This allows the parameters to be configured using vxprj or Workbench. However, you are not required to do this.

Device configuration sections must be placed inside the chosen sub-group in **itl_generic.dts**. For example, the customizable settings for the driver **vxblaNs16550Sio** are:

```
chosen{
  serialConfig{
    serialConfig0{
      node = <0>;
      clock-frequency = <DRV_SIO_IA_NS16550_INSTANCE_CLK_FREQ>;
      baud-rate = <DRV_SIO_IA_NS16550_INSTANCE_BAUD_RATE>;
      fifo-size = <DRV_SIO_IA_NS16550_INSTANCE_FIFO_LEN>;
      reg-delta = <DRV_SIO_IA_NS16550_INSTANCE_REG_INTERVAL>;
      };
```

The **itl_generic.dts** file defines 1 serial port and 1 serial configuration for node 0. The board has 2 serial ports discovered by ACPI.

Each driver is responsible for validating configuration parameters that are present in the device tree source (DTS). If no configuration parameters for a specific instance (node number) of a driver are present, the driver fails to attach to the device. This allows the DTS to define the default device nodes typically found on an Intel system, but not use them if they are found by ACPI instead.

The DRV_BUS_ACPI_ROOT driver creates a VxBus driver node named **PNP0501:0**, where **:0** is the node number. The serial port driver's attach function configures the device based on the serial configuration for node 0. The DRV_BUS_ACPI_ROOT driver then creates **PNP0501:1**. However, the serial driver's attach function returns a failure because the DTS file does not specify a configuration

for node 1. After all ACPI devices are processed, the DRV_BUS_FDT_ROOT driver walks the DTS file and enumerates all device nodes. It creates device node **PNP0501:2**, which also fails to attach because there is no serial port configuration for node 2.

In this example, the VxWorks Image Project (VIP) parameter *DRV_SIO_IA_NS16550_INSTANCE_CLK_FREQ* is used to select *clock-frequency*. This gives the option to set this parameter from vxprj or Workbench.

The parameter *DRV_SIO_IA_NS16550_INSTANCE_FIFO_LEN* also sets the UART FIFO size. To support this configuration option, the **vxbNs16550Sio** driver has been written with FIFO size detection logic. The FIFO size is set to 16 bytes if the hardware UART supports FIFO functionality, otherwise it is just set to 1.

Usually there are multiple COM ports enabled in the target BIOS (ACPI table). While *DRV_SIO_IA_NS16550_INSTANCE_0* is set by default, subsequent instances *DRV_SIO_IA_NS16550_INSTANCE_n* will be defined in the VIP.

According to the ACPI table scanning order of serial COM devices in the BIOS, the CONSOLE_NAME VIP parameter can be set to **/ttyS0**, **/ttyS1**, **/ttyS2**, or **/ttyS3**.

5. Example Device Driver for the itl_generic BSP

Use the sample driver provided to better understand the recommended code flow for a device that can be discovered by ACPI and flattened device tree (FDT).

The driver does the following:

1. The probe function matches a VxBus device based on the nodes name. In this case, the ACPI HID PNP0000 or the node name PNP0000 from the **itl_generic.dts** file.
2. The attach function searches for configuration data that matches the instance of the device. If no configuration data is found, the function returns **ERROR**.
3. The attach function allocates IO and IRQ resources and initializes interrupts.

The sample driver is as follows:

```
/* SampleDriver.c  */

/*

FDT NODE DECLARATION

When the device node is declared in the DTS file it must be named PNP0000 and
contain the following parameters.

sample0: PNP0000
 {
 io = <DRV_SAMPLE_INSTANCE_0_BASE_ADDR 0x8>;
 interrupts = <DRV_SAMPLE_INSTANCE_0_INT_VEC
               DRV_SAMPLE_INSTANCE_0_INT_TYPE
               DRV_SAMPLE_INSTANCE_0_INT_IOAPIC_PIN
               DRV_SAMPLE_INSTANCE_0_TRIGGER
               DRV_SAMPLE_INSTANCE_0_POLARITY
 };

FDT NODE CONFIGURATION

When the device node is discovered by either ACPI or FDT a node configuration
is required. The node configuration for this device is in the following format:
```

```
example:

 sampleConfig0{
 node = <0>;
 clock-frequency = <DRV_SAMPLE_INSTANCE_0_CLK_FREQ>;
 };

*/

/* includes */

#include <vxWorks.h>
#include <vsbConfig.h>
#include <intLib.h>
#include <hwif/vxBus.h>
#include <subsys/int/vxbIntLib.h>
#include <hwif/util/vxbAccess.h>
#include <hwif/util/vxbResourceLib.h>
#include <hwif/vxbus/vxbLib.h>

#include <vxFdtLib.h>
#include <vxbAcpiBus.h>

/* externs */

IMPORT int fdt_path_offset(const void *fdt, const char *path);

/* forward declarations */

LOCAL STATUS sampleProbe (VXB_DEV_ID pDev);
LOCAL STATUS sampleAttach (VXB_DEV_ID pDev);

LOCAL VXB_DRV_METHOD sampleMethodList[] =
    {
    /* DEVICE API */

    {VXB_DEVMETHOD_CALL(vxbDevProbe), sampleProbe},
    {VXB_DEVMETHOD_CALL(vxbDevAttach), sampleAttach},
    VXB_DEVMETHOD_END
    };

/* driver registration data structure */

VXB_DRV fdtSampleDrvRegistration =
    {
    { NULL },                           /* vxbNode */
    "PNP0000",                          /* Name */
    "Sample driver",                    /* Description */
    VXB_BUSID_FDT,                      /* Class */
    0,                                  /* Flags */
    0,                                  /* Reference count */
    sampleMethodList                    /* Method table */
    };

VXB_DRV acpiSampleDrvRegistration =
    {
```


WNDRVR

```
    { NULL },                           /* vxbNode */
    "PNP0000",                          /* Name */
    "Sample driver",                    /* Description */
    VXB_BUSID_ACPI,                     /* Class */
    0,                                  /* Flags */
    0,                                  /* Reference count */
    sampleMethodList                    /* Method table */
    };

/*
 * Register two instances of this driver, one as a child of the ACPI bus
 * and one as a child of the FDT bus.
 */

VXB_DRV_DEF(fdtSampleDrvRegistration)
VXB_DRV_DEF(acpiSampleDrvRegistration)

LOCAL STATUS sampleProbe
    (
    struct vxbDev * pDev /* Device information */
    )
    {

    /*
     * vxbDevMatch matches the vxBus node name to the name set in the VXB_DRV
     * registration. Since this device can be discovered by ACPI, the HID reported by
     * ACPI, the name of the device in the DTS file, the FDT VXB_DRV registration,
     * and the ACPI VXB_DRV registration must all match
     */

    return vxbDevMatch(pDev);
    }

LOCAL STATUS sampleAttach
    (
    struct vxbDev * pDev /* Device information */
    )
    {
    VXB_RESOURCE_ADR  * pResAdr = NULL;
    VXB_RESOURCE      * pResIo;
    VXB_RESOURCE      * pResIrq;

    int len;
    int devoffset;
    const void * prop;
    int     offset;
    void * gpDtb = NULL;
    UINT32 unit;
    UINT32 clockFreq;

    /* Get the device instances unit number */

    unit = vxbDevUnitGet(pDev);

    /* Get the base address of the FDT compiled blob (DTB) */

    gpDtb = vxFdtDtbAddressGet();
```

```
/* Search for sample node configuration */

offset = fdt_path_offset (gpDtb, "/chosen/sampleConfig");

/* If no driver configuration found, return error */

if (offset < 0)
    return ERROR;

/* Find sample configuration node matching device node */

for (devoffset = vxFdtFirstSubnode (offset); devoffset > 0;
      devoffset = vxFdtNextSubnode (devoffset))

     {
         prop = vxFdtPropGet (devoffset, "node", &len);
         if (prop != NULL)
             {
             if (unit == vxFdt32ToCpu (*(UINT32 *)prop))
                 break;

             }
     }

/* return ERROR if no configuration found */

if (devoffset <= 0)
    return ERROR;

prop = (void*)vxFdtPropGet (devoffset, "clock-frequency", &len);

if ((NULL == prop) ||(sizeof (UINT32) != len))
    return ERROR;

/* Set clock-frequency */

clockFreq = vxFdt32ToCpu (*(UINT32 *)prop);

/* Allocate IO Resource */

pResIo = vxbResourceAlloc(pDev, VXB_RES_IO, 0);

if(pResIo == NULL)
    return ERROR;

pResAdr = (VXB_RESOURCE_ADR *)pResIo->pRes;

if (pResAdr == NULL)
    return ERROR;

/* Allocate IRQ resource */

pResIrq = vxbResourceAlloc(pDev, VXB_RES_IRQ, 0);

if (pResIrq == NULL)
    return ERROR;
```

```
    /* Enable interrupts */

    if (vxbIntConnect(pDev, pResIrq, pFunc, pArg) != OK)
        return ERROR;

    if (vxbIntEnable(pDev, pResIrq) != OK)
        return ERROR;

    return OK;
    }
```

6. Example FDT File for the itl_generic BSP

Use the sample flattened device tree (FDT) file for the itl_generic BSP as a model for your development and to better understand how FDT is used to configure devices in an Intel-based system.

Given the sample driver in Example Device Driver for the itl_generic BSP on page 69, the corresponding entries in the **itl_generic.dts** file are similar to the following:

```
sample0: PNP0000
 {
 io = <DRV_SAMPLE_INSTANCE_0_BASE_ADDR 0x8>;
 interrupts = <DRV_SAMPLE_INSTANCE_0_INT_VEC
               DRV_SAMPLE_INSTANCE_0_INT_TYPE
               DRV_SAMPLE_INSTANCE_0_INT_IOAPIC_PIN
               DRV_SAMPLE_INSTANCE_0_TRIGGER
               DRV_SAMPLE_INSTANCE_0_POLARITY
 };
```

```
chosen{

        sampleConfig{

            sampleConfig0{
                node = <0>;
                clock-frequency = <DRV_SAMPLE_INSTANCE_0_CLK_FREQ>;
                };
          }
       }
```

When the DRV_BUS_ACPI_ROOT component is included in your VxWorks image project (VIP) image and ACPI tables reported by the BIOS declare there are 4 instances of the PNP0000, the first PNP0000 device (**PNP0000:0**) discovered by DRV_BUS_ACPI_ROOT uses the first **sampleConfig0 (node = <0>)** declared in the **itl_generic.dts** file. The remaining 3 PNP0000 devices (**PNP0000:1**, **PNP0000:2**, **PNP0000:3**) return an error during the sample driver's attach function and are not available for use in VxWorks.

The DRV_BUS_FDT_ROOT then creates a **PNP0000:4** node but returns an error during the sample driver's attach function because it expects to find a **sampleConfig4 (node = <4>)** present in the device tree source (DTS) file.

In order for VxWorks to utilize the 3 remaining PNP0000 devices reported by ACPI or the static device declared in the DTS file, you must update the **itl_generic.dts** file with additional configuration entries (for example, **sampleConfig1 (node = <1>)** and so forth).

If the component DRV_BUS_ACPI_ROOT is not included in the VIP image, the DRV_BUS_FDT_ROOT component creates a **PNP0000:0** node and the sample driver's attach function uses the **sampleConfig0 (node = <0>)** in the DTS file for configuration.

WNDRVR

In order for VxWorks to utilize additional PNP0000 devices, you must update the **itl_generic.dts** file with additional device entries (**sample1: PNP0000**) as well as configuration entries (for example, **sampleConfig1 (node = <1>)** and so forth).

7. BOARD SPECIFIC INFORMATION

1. Lava Canyon CRB Specific Issues

Use Telnet Instead of PC Console

Issues exist with the onboard Lava Canyon CRB serial port, and so *lava_canyon_readme.md* directs the use of **INCLUDE_PC_CONSOLE** and USB keyboard components. If this is not possible, the alternative strategy is to add the telnet server to the VIP instead.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_IPTELNETS
```

Once VxWorks boots, you must telnet from your workstation to the VxWorks target to access to the kernel shell.

NVRAM

There is no onboard NVRAM. To recreate NVRAM capability, you must insert a USB flash drive or a SATA drive into the board. You must also include the relevant component, which is either **INCLUDE_SYS_WARM_USB** or **INCLUDE_SYS_WARM_AHCI**.

USB

EHCI and XHCI have been verified on the Lava Canyon Haswell board. For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_EHCI_INIT
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USB_XHCI_HCD_INIT
```

2. Cedar Rock, Norco and Bay Trail CRB Specific Issues

When you boot VxWorks from any boot loader other than the boot application, you must reset the USB UHCI controller after system startup. Refer to *cedar_rock_readme.md*, *baytrail_readme.md* or *norco_readme.md* for details. If you use the boot application to load VxWorks, this additional step is not necessary.

3. Bayley Bay CRB Specific Issues

1. The target board AMI BIOS produces a valid MP table, so you need not add INCLUDE_ACPI_BOOT_OP to the VIP.
2. Intel Hyper-Threading technology is not supported on Bayley Bay target boards.

4. NITX-315 CRB Specific Issues

Some NITX-315 BIOS revisions set the serial FIFO control register incorrectly during ACPI initialization. This causes garbled output from the target serial port.

The solution is to reset the serial FIFO control register to a known, good value. To do this, add the following components to the VIP:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_SIO_PCI_NS16550
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_NITX_315_BF
```

5. Shumway CRB Specific Issues

Incomplete MP Table

The BIOS on the Shumway board requires the component **INCLUDE_ACPI_BOOT_OP** to be included in the multi-stage boot application to generate a correct MP table.

PCI Bus Controller

On the Shumway CRB board, the first PCI bus controller starts from bus 0 and the second PCI bus controller starts from bus 128. However, they are not bridge connected.

The Shumway CRB board revision B has a PCI-to-PCI bridge controller on PCI bus 128, which bridges to bus 130. However, the same controller on the revision A board is not configured as a PCI-to-PCI bridge. In order to search the bus 130 or above on the revision A board, you must create a third VxBus PCI bus controller device instance with the start at bus 130.

The default configuration for a boot application and VIP is to use the **DRV_BUS_ACPI_ROOT** driver. This driver discovers and enumerates all PCI devices reported by ACPI even if they are not bridge connected.

6. Camelback Mountain CRB Specific Issues

UART

The Camelback Mountain 400 CRB uses the Broadwell-DE UART for V0/V1 stepping (ES2) silicon and must be set with a different UART base clock frequency of 153600 Hz. This equates to an SIO clock frequency of 2457600 Hz.

Not all baud rate combinations can be derived from the clock provided on the Camelback Mountain 400 for V0/V1 stepping (ES2). Therefore, a baud rate of 38400 is selected with the UART base clock ((2457600/16) / 4) = 38400).

Table 1. UART Base Clock Rate Settings

Desired Baud Rate	UART base clock 115200	UART base clock 153600	UART base clock 172800
115200	1	*	*
57600	2	*	3
38400	3	4	*
28800	4	*	6
19200	6	8	9
14400	8	*	12

Desired Baud Rate	UART base clock 115200	UART base clock 153600	UART base clock 172800
9600	12	16	18

Note: * indicates the particular baud rate cannot be achieved with the given base clock rate.

PC Console

No PS2 keyboard is available on the target board. Therefore, when adding **INCLUDE_PC_CONSOLE** to the VIP, you must also add the USB keyboard component **DRV_KBD_USB**.

Serial Console

In order to configure the target with a serial console instead of a video console, modify the VIP as follows. Notice that a USB keyboard is required when using a serial console:

```
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component remove INCLUDE_PC_CONSO
LE
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip parameter set DRV_SIO_IA_NS16550_
INSTANCE_0_TRIGGER 2
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip parameter set DRV_SIO_IA_NS16550_
INSTANCE_0_POLARITY 2
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip parameter set DRV_SIO_IA_NS16550_
INSTANCE_CLK_FREQ 1843200
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip parameter set CONSOLE_BAUD_RATE 1
15200
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add INCLUDE_EHCI_INIT I
NCLUDE_OHCI_INIT INCLUDE_UHCI_INIT
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add DRV_KBD_USB INCLUDE
_USB_GEN2_KEYBOARD_INIT
C:\WindRiver\workspace\itl_generic_nehalem_kernel_smp> vxprj vip component add INCLUDE_EVDEV_LIB_K
BD_SHELL_ATTACH
```

Port 60/64 Emulation

Port 60/64 emulation must be disabled if:

- the VxWorks kernel hangs when the VIP is configured with **INCLUDE_PC_CONSOLE**.
- the target board takes too long to boot with a multi-stage boot application that is configured with **INCLUDE_PC_CONSOLE**.

To disable port 60/64 emulation, refer to either *camel_200_300_readme.md* or *camel_400_readme.md* for details.

7. Bearlake CRB Specific Issues

MC146818 Real-Time Clock Specific Issues

The VxBus driver source file is **vxbMc146818Rtc.c**.

When adding the component **INCLUDE_MC146818_AUX_CLK** or **DRV_TIMER_MC146818**, you must also remove the component **DRV_BUS_ACPI_ROOT**. This works around an ACPI initialization issue with the driver where interrupt information is not properly configured when using ACPI information.

HPET Configuration Issues

By default, HPET timers use MSI interrupts. However, this board does not support HPET MSI interrupts. Consequently, the multi-stage boot application or VxWorks Image Project (VIP) that uses HPET timers must always set the corresponding **DRV_TIMER_IA_HPET_INSTANCE_n_INT_TYPE** to 1. For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set DRV_TIMER_IA_HPET_INSTANCE_1_INT_TY
PE 1
```

Timestamp Driver

An invariant TSC is unavailable on the Bearlake CRB. This means that the TSC cannot be used with the timestamp driver. Instead, the HPET timer is used with the timestamp driver on Bearlake.

8. Oxbow Hill CRB Specific Issues

Using the EFI Console Instead of the Serial Port

If you opt not to use the target serial port for the VxWorks console, you must enable both the EFI console and USB keyboard support on the target instead.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_PC_CONSOLE
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_CONSOLE_EFI
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USB_XHCI_HCD
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USB_XHCI_HCD_INIT
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USB_GEN2_KEYBOARD
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_USB_GEN2_KEYBOARD_INIT
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_EVDEV_LIB_KBD_SHELL_ATTACH
```

Issues With the Boot Application

The boot application is not supported by Wind River on this board. Wind River recommends you use the UEFI boot loader instead.

However, the boot application may work for some configurations. To use the boot application, you must modify the **RAM_LOW_ADRS** BSP parameter. This is the destination virtual address that the boot application image is copied to before the system executes the boot application image. Refer to Modifying BSP Parameter RAM_LOW_ADRS on page 80 for instructions on how to change **RAM_LOW_ADRS**, and Boot Application Memory Map Details on page 79 for an explanation of the reason for the change.

SD Card Support

To enable SD card support on Oxbow Hill, follow the instructions given in SDHC/MMC Configuration on page 56 and also add the component **DRV_PINMUX_IA** to the VxWorks Image Project (VIP). For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_PINMUX_IA
```

Other Issues

1. Only the UEFI boot loader is supported for the Oxbow Hill CRB. When booting the target from the UEFI boot loader, expect a 15 second delay before the VxWorks banner appears.

2. You must not enable CSM mode in the target board BIOS. If you do, then when pressing keyboard keys the target will generate double characters.

> Note: If you enable CSM mode, the only way to disable it again is to re-flash the BIOS in target flash memory.

Boot Application Memory Map Details on page 79
Modifying BSP Parameter RAM_LOW_ADRS on page 80

8.1. Boot Application Memory Map Details

Memory Map Configuration

As the board comes out of reset, the initial BSP boot stages setup a static MMU table so that the BSP can configure the target board to run VxWorks. The BSP is located in the BSP directory in the source file **romInit.s**.

32-Bit Memory Map

Table 1. Physical to Virtual Memory Mappings on Oxbow Hill

Address Range Size	Physical Address Range		Virtual Address Range
512 MB	0x00000000 - 0x1fffffff	IS MAPPED TO	0xe0000000 - 0xffffffff
512 MB	0x00000000 - 0x1fffffff	IS MAPPED TO	0x00000000 - 0x1fffffff

The default value of **RAM_LOW_ADRS** is virtual address 0xf0008000. The BSP static MMU map configuration maps this virtual address onto physical address 0x10008000 (256 MB + 8 KB).

On Oxbow Hill, the physical address 0x10008000 is not available. The entry starting at 0x100000 (1MB) in E820 is approximately 239 MB. Any attempt to copy to physical address 0x10008000 will fail on this board.

The solution is to reduce the setting of **RAM_LOW_ADRS** from virtual address 0xf0008000 to 0xe8008000. This references physical address 0x08008000 (128 MB + 8 KB) which is addressable memory.

WNDRVR

64-Bit Memory Map

Table 2. Physical to Virtual Memory Mappings on Oxbow Hill

Address Range Size	Physical Address Range		Virtual Address Range
2048 MB	0x00000000_00000000 - 0x00000000_80000000	IS MAPPED TO	0xffffffff_80000000 - 0xffffffff_ffffffff
2048 MB	0x00000000_00000000 - 0x00000000_80000000	IS MAPPED TO	0x00000000_00000000 - 0x00000000_80000000

The default value of **RAM_LOW_ADRS** is virtual address 0xffffffff_90008000. The BSP static MMU map configuration maps this virtual address onto physical address 0x00000000_10008000 (256 MB + 8 KB).

On Oxbow Hill, the physical address 0x00000000_10008000 is not available.

The solution is to reduce the setting of **RAM_LOW_ADRS** from virtual address 0xffffffff_90008000 to 0xffffffff_88008000. This references physical address 0x00000000_08008000 (128 MB + 8 KB) which is addressable memory.

Parent topic: Oxbow Hill CRB Specific Issues on page 78

8.2. Modifying BSP Parameter RAM_LOW_ADRS

About This Task

These steps describe how to modify the **RAM_LOW_ADRS** parameter to enable the boot application for the Oxbow Hill CRB.

Procedure

1. Locate the file **20bsp.cdf** in the BSP directory.
2. Open in an editor and locate the definition of **RAM_LOW_ADRS**:

```
Parameter RAM_LOW_ADRS {
    NAME        Runtime kernel load address
```

3. Change the definition of **RAM_LOW_ADRS** as follows:

```
Parameter RAM_LOW_ADRS {
    NAME        Runtime kernel load address
#ifdef _WRS_CONFIG_LP64
    DEFAULT     (INCLUDE_BOOT_RAM_IMAGE)::(0xffffffff84008000) \
                (INCLUDE_BOOT_APP)::(0xffffffff88008000)       \
                0xffffffff80408000
#else
    DEFAULT     (INCLUDE_BOOT_RAM_IMAGE)::(0xe4008000) \
                (INCLUDE_BOOT_APP)::(0xe8008000)       \
                0xe0408000
#endif /@ _WRS_CONFIG_LP64 @/
}
```


WNDRVR

Parent topic: Oxbow Hill CRB Specific Issues on page 78

9. Greencity and Allagash CRB Specific Issues

HPET Configuration Issues

By default, HPET timers use MSI interrupts. However, this board does not support HPET MSI interrupts. Consequently, the multi-stage boot application or VxWorks Image Project (VIP) that uses HPET timers must always set the corresponding **DRV_TIMER_IA_HPET_INSTANCE_n_INT_TYPE** to 1. For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set DRV_TIMER_IA_HPET_INSTANCE_1_INT_TY
PE 1
```

10. Harcuvar and Car Creek CRB Specific Issues

Using the EFI Console Instead of the Serial Port

If you opt not to use the target serial port for the VxWorks console, you must enable both the EFI console and USB keyboard support on the target instead.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add INCLUDE_PC_CONSOLE
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_CONSOLE_EFI
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component add DRV_KBD_USB
```

APIC Issue

The Harcuvar and Car Creek CRB supports extended xAPIC (x2APIC) architecture. This makes xAPIC architecture backwards compatible. While the board will boot with x2APIC enabled, VxWorks does not support this mode and will not run. Refer to either *harcuvar_readme.md* or *car_creek_readme.md* for details on how to disable x2APIC in the BIOS.

11. AMD Steppe and Bald Eagle Board Specific Issues

1. Do not include the **DRV_CONSOLE_EFI** component into project builds because the UEFI VGA driver does not function.
2. Do not use the 32-bit EFI boot loader because it is not functioning. This means you must not use the file **BOOTIA32.EFI** on this target. You must always use **BOOTIA64.EFI** instead.
3. MSI interrupts are not supported by the HPET timer, and so the HPET timer interrupt configuration should be changed to use IRQ instead. For example, when creating a VIP using the HPET that is intended for this board, you must add the following components and set the parameters like this:

```
C:\WindRiver\workspace\vip32_smp> vxprj component add INCLUDE_HPET_SYS_CLK
C:\WindRiver\workspace\vip32_smp> vxprj component add DRV_INTCTLR_IOAPIC
C:\WindRiver\workspace\vip32_smp> vxprj parameter set DRV_TIMER_IA_HPET_INSTANCE_0_INT_IOAPIC_P
IN 22
C:\WindRiver\workspace\vip32_smp> vxprj parameter set DRV_TIMER_IA_HPET_INSTANCE_0_INT_TYPE 1
```

12. Kabylake CRB Specific Issues

When booting VxWorks on the Kabylake CRB, you must always change the serial IRQ mode in the BIOS from **Quiet** to **Continuous**. Refer to *kabylake_readme.md* for details.

13. Skylake CRB Specific Issues

UART

Pre-production Skylake CRBs RVP8/RVP11 have ACPI data that contains incorrect trigger and polarity values. This causes VxWorks serial driver initialization failure. Because the ACPI data is incorrect for these CRBs, ACPI must not be used. Instead, FDT must be used to manually configure the board. Remove the component **DRV_BUS_ACPI_ROOT** to remove ACPI support from the VIP.

> Note: This behavior has only been seen on pre-production boards. It has not been observed by Wind River on production boards and BIOS.

Also, the serial IRQ mode must be changed from Quiet to Continuous on the RVP8/RVP11 boards. Refer to *skylake_readme.md* for details.

PC Console

When you include support for the PC console, you must also add the EFI console and USB keyboard drivers:

- **INCLUDE_PC_CONSOLE**
- **DRV_CONSOLE_EFI**
- **DRV_KBD_USB**

Known Issues

Intel is investigating a TSC read delay issue on this board. The work around is to disable Enhanced C States in the board's BIOS. Refer to *skylake_readme.md* for details.

LPSS UART

Intel introduced the Low Power Sub System (LPSS) in the Skylake processor generation. The LPSS is a clustered module that contains devices such as the UART, SPI, I2C. Normally, the LPSS is used as a non-sleep module that continues to function even after the whole system switches to low-power or sleep mode, but it is also used as a conventional device for additional UART, SPI, or I2C channels.

Only LPSS UART is supported in this release.

14. CoffeeLake and CoffeLake Refresh CRB Specific Issues

PC Console

When the **INCLUDE_PC_CONSOLE** component is included into the VIP, **DRV_CONSOLE_EFI** must also be included.

WNDRVR

Bootloader

Both the UEFI bootloader (**BOOTX64.EFI**) and GRUB are supported as bootloaders on CoffeeLake and CoffeLake Refresh CRB.

Note: UEFI bootloader (**BOOTIA32.EFI**), BIOS bootloader and the Multi-Stage Bootloader are not supported.

Known Issues

The reading TSC delay exists on these CRBs if C-State is enabled in the BIOS. The workaround to this issue is to disable C-States in the BIOS.

WNDRVR

8. GENERAL BSP INFORMATION

1. Supported BSP Make Targets

This BSP supports the following target image from vxprj and Workbench:

Table 1. Data Model VSB Options

Target Image	Description
vxWorks_romCompress.bin	The compressed second stage of the multi-stage boot application and also VxWorks ROM image
vxStage1Boot.bin	First stage of the multi-stage boot application
vxStage1Boot.pxe	First stage of the multi-stage boot application used to boot target board from PXE boot.
vxWorks with **vxWorks.sym**	Standard VxWorks RAM image

Note: Any image not listed in this table may not be tested with this BSP.

2. Special BSP Functions

The following API is implemented specifically as part of this BSP and are therefore available to be used in user applications. They are written in assembly language code and located in **sysALib.s**.

Table 1. Data Model VSB Options

API Function	Description of Function
sysInByte( )	Input one byte from I/O space
sysInWord( )	Input one word from I/O space
sysInLong( )	Input one long-word from I/O space
sysOutByte( )	Output one byte to I/O space
sysOutWord( )	Output one word to I/O space
sysOutLong( )	Output one long-word to I/O space
sysInWordString( )	Input word string from I/O space
sysInLongString( )	Input long-word string from I/O space
sysOutWordString( )	Output word string to I/O space

API Function	Description of Function
sysOutLongString( )	Output long-word string to I/O space

3. BSP Build Models

This BSP supports four distinct build models.

Table 1. Data Model VSB Options

Build Model	Usage
Uniprocessor (UP)	The VxWorks image is built to run on CPU 0 only, only one instance of VxWorks is running
Symmetric Multiprocessor (SMP)	The VxWorks image is built to run on a CPU set, starting at CPU 0
Boot Application	A ROM or RAM based UP boot loader for loading VxWorks images from CPU 0. Can load UP and SMP images.

Under the SMP build model, you can enable between 1 and 32 CPUs. The total number of CPUs (physical plus hyper-threaded CPUs) must not exceed 32. The selected CPUs are allocated contiguously starting at CPU 0 (the base CPU).

The number of CPUs assigned to a VxWorks image is controlled by the project parameter **VX_SMP_NUM_CPUS**. This parameter defaults to the number of CPUs supported by the selected processor type.

Modifying the Number of CPUs on page 85

3.1. Modifying the Number of CPUs

Before You Begin

These steps assume you have created a Symmetric Multiprocessr (SMP) VxWorks Image Project (VIP). Refer to Creating a VxWorks Image Project on page 13.

Procedure

1. Change to the VIP directory.
2. Adjust the number of CPUs supported by VxWorks.

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set VX_SMP_NUM_CPUS 2
```

Parent topic: BSP Build Models on page 85

4. Memory Maps

For BSP memory map information, see the VxWorks memory layout information in the *VxWorks 7 Intel Architecture Guide:VxWorks Memory Layout for IA-32 Systems*, *VxWorks 7 Intel Architecture Guide:VxWorks Memory Layout for Intel 64 Systems* and the *VxWorks 7 Platform Programmer's Guide:VxWorks 7 Memory Model*.

9. KNOWN BSP ISSUES

1. BIOS and MP Table

BIOS constraints sometimes limit the number of cores available when VxWorks is configured to boot using the MP table constructed by the BIOS.

On some boards, incorrect MP table entries can prevent the use of legacy interrupts for devices such as the GEI network driver. In these cases, you must use MSI interrupts instead. This requires ACPI.

The Intel architecture BSPs are dependent on an external third-party for the initial configuration/description of the existing hardware environment. By default, Wind River uses the BIOS setup utility because it is the most universal. This utility gathers information and provides it in the form of the MP table.

When an issue with the MP table occurs, and until a BIOS update is available from the hardware vendor, Wind River recommends that you bypass the BIOS MP table construction and create the MP table with the information gathered using ACPI. You can do this by configuring the BSP to use the **INCLUDE_ACPI_BOOT_OP** component.

Wind River does not provide BIOS updates.

2. Reboot

Sometimes a board will not reboot properly if a reboot command is issued prior to the completion of the AHCI/ICH driver initialization process. This occurs for both boot applications and with the VxWorks kernel.

On some boards, SMP VxWorks images will not warm reboot properly when loaded by a boot application configured with **INCLUDE_PC_CONSOLE** (or VGA console) support. One workaround is to use a boot application configured to use the serial console. Another workaround is to disable the legacy USB SMI interrupt mode in the boot application. For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component remove INCLUDE_LEGACY_USB_INT_MODE
```

If available, you can use a PS2 keyboard. When the legacy USB SMI interrupt mode is turned off, PS2 keyboards are disabled.

3. PCI Express Gen 2

The PCI Express slots in the target hardware board are PCI Express Gen2. There are some compatibility issues with pre-Gen2 PCI Express cards.

4. MTRR

The effective memory type depends on the *MTRR*, *PCD* and *PWT* settings.

Table 1. Data Model VSB Options

MTRR Mem Type	PCD Value	PWT Value	Effective Mem Type
UC	X	X	UC
WC	0	0	WC
	0	1	WC
	1	0	WC
	1	1	UC
WT	0	X	WT
	1	X	UC
WP	0	0	WP
	0	1	WP
	1	0	UC
	1	1	UC
WB	0	0	WB
	0	1	WT
	1	X	UC

The MTRR memory types and their properties are defined as follows:

Table 2. Data Model VSB Options

Mnemonic	Cacheable in L1 and L2 Caches	Writeback Cacheabe	Allows Speculative Reads	Memory Ordering Model
UC	No	No	No	Strong Ordering
WC	No	No	Yes	Weak Ordering
WT	Yes	No	Yes	Speculative Processor Ordering
WP	Yes for reads, No for writes	No	Yes	Speculative Processor Ordering
WB	Yes	Yes	Yes	Speculative Processor Ordering

5. Memory Auto-Size

Memory Auto Size Using E820 BIOS Memory Map Information

Using the 0x15 interrupt with function code 0xe820, the physical RAM memory map can be retrieved from the BIOS. The data returned is a set of descriptors containing the memory range start address, length, and type. The data is acquired while in 16-bit REAL mode during cold boot in the boot application, and stored in a fixed location in reserved memory. This data is in turn consumed by the VxWorks image during startup to dynamically adjust the top of memory. Additional memory segments beyond the one located at *LOCAL_MEM_LOCAL_ADRS* are automatically mapped to the MMU.

Memory auto size using BIOS E820 data is enabled by the component **INCLUDE_BIOS_E820_MEM_AUTOSIZE**, which is included by default. For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip component remove INCLUDE_BIOS_E820_MEM_AUTOSIZE
```

Memory Auto Size Using Multiboot Compliant Boot Loader

When GRUB is used to load VxWorks, you can use the multiboot header information to determine memory size. Memory auto-size using multiboot header information is enabled by the **INCLUDE_MULTIBOOT_MEM_AUTOSIZE** component, which is enabled by default.

If both auto-size methods are enabled, the multiboot header is checked first, and if it is not present (if not booting from a multiboot complaint boot loader, or if the multiboot header information is not valid), the BIOS E820 method is used. To completely disable all memory auto-sizing, remove both the **INCLUDE_MULTIBOOT_MEM_AUTOSIZE** and **INCLUDE_BIOS_E820_MEM_AUTOSIZE** components.

When memory auto-sizing is disabled, the system RAM size is defined by the **SYSTEM_RAM_SIZE** parameter.

Only the boot application scans the physical RAM memory map. VxWorks uses the memory map already scanned by the boot application. This memory mapping process will not be done prior to VxWorks boot if VxWorks is not loaded by the boot application (such as when loaded by GRUB). In such cases, only the memory specified statically in the *sysPhysRamDesc[]* structure of **usrItlCommon.c** is available. To use all available RAM, disable memory auto-sizing and properly configure the *sysPhysRamDesc[]* structure. For more information, refer to the *VxWorks 7 Platform Programmer's Guide:VxWorks 7 Memory Model*.

> Note: The *sysPhysRamDesc[0]* RAM is used to allocate the spaces for the DMA32 partition, user memory, ED&R persistent memory, boot ROM, and global mapping area which includes the kernel image, kernel proximity heap, and so forth. To ensure that the system can work normally, the sum of all space allocated must be smaller than the *sysPhysRamDesc[0]* RAM.

6. Error Detection and Reporting (ED&R) Default Memory Size

ACPI device enumeration can exhaust the ED&R memory pool on some boards. The memory pool size for ED&R is defined by the *MEDR_POOL_SIZE* parameter. The default size is **0x100000**, which is defined in **02mem_edr.cdf**.

To increase the memory pool size, set the **MEDR_POOL_SIZE** parameter in your VxWorks Image Project (VIP). For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set MEDR_POOL_SIZE 0x400000
```

7. ACPI Maximum Loop Iteration Counter

The maximum loop iteration counter allows the ACPI interpreter to escape a possible infinite loop condition resulting from poorly written ACPI Machine Language (AML).

Because the default value of **AcpiGbl_MaxLoopIterations** is too large, the board can take hours to boot if the AML is written poorly. To improve this situation, you can decrease the value of the **AcpiGbl_MaxLoopIterations** parameter.

To decrease the ACPI maximum loop iteration counter, set the **ACPI_MAX_LOOP_COUNTER_VX** parameter in your VIP. For example:

```
~/home/user/WindRiver/workspace/myVIP$ vxprj vip parameter set ACPI_MAX_LOOP_COUNTER_VX 128
```

8. Shared Memory

Hardware specific shared memory is not applicable to this BSP.

9. Advanced Vector Extensions 512 (AVX-512)

AVX-512 is not supported by VxWorks. The kernel does not save the 512-bit wide SIMD registers (**ZMM0**-**ZMM31**) on a task context switch.

10. REFERENCES

1. See Also

- VxWorks 7 Configuration and Build Guide
- VxWorks 7 Intel Architecture Guide
- VxWorks 7 Platform Programmer's Guide
- VxWorks Boot Loader User's Guide
- VxWorks 7 BSP and Driver Guide
- Helix Platform Configuration and Build Guide

2. Bibliography

- *Intel Architecture Software Developer's Manual, Volume 1: Basic Architecture*
- *Intel Architecture Software Developer's Manual, Volume 2: Instruction Set Reference*
- *Intel Architecture Software Developer's Manual, Volume 3: System Programming Guide*
- *AP-485, Intel Processor Identification and the CPUID Instruction*
- *PCI System Architecture, Fourth Edition, Addison-Wesley, 1999, ISBN 0-201-30974-2*
- *Preboot Execution Environment (PXE) Specification, Intel*